

Stop When Reasoning Converges: Semantic-Preserving Early Exit for Reasoning Models

Dehai Min^{1,*,†}, Giovanni Vaccarino^{1,4,*}, Huiyi Chen¹,
Yongliang Wu³, Gal Yona², Lu Cheng¹

¹University of Illinois Chicago ²Google Research

³University of Illinois Urbana-Champaign ⁴Politecnico di Milano

Abstract

Large Reasoning Models (LRMs) achieve strong performance by generating long chains of thought (CoT), but often overthink, continuing to reason after a solution has already stabilized and thereby wasting tokens and increasing latency. Existing inference-time early-exit methods rely primarily on answer-level signals, such as confidence or trial-answer consistency, to decide when to stop. However, these signals mainly reflect answer readiness rather than reasoning convergence: they may trigger before the model has finished exploring or self-correcting, causing premature exits that can degrade final-answer accuracy and leave the retained reasoning chain semantically incomplete. We identify reasoning-level semantic redundancy as a complementary signal for semantic-preserving early exit: when successive steps no longer add novel progress and instead revisit established conclusions, the reasoning trajectory has likely converged. Building on this insight, we propose PUMA, a plug-and-play framework that combines a lightweight Redundancy Detector with answer-level verification. The detector flags semantically redundant candidate exits, while verification confirms whether stopping is safe, allowing PUMA to remove redundant continuation while preserving both answer accuracy and a coherent reasoning prefix. Across five LRMs and five challenging reasoning benchmarks, PUMA achieves 26.2% average token reduction while preserving accuracy and retained CoT quality. Additional experiments on code generation, zero-shot vision-language reasoning, and learned stopping-policy internalization further demonstrate that reasoning-level redundancy is a robust, transferable, and learnable signal for efficient reasoning. Our code is available at <https://github.com/giovanni-vaccarino/PUMA>.

1 Introduction

Recent Large Reasoning Models (LRMs) such as DeepSeek-R1 [1] and OpenAI o1 [2] achieve strong performance by generating long chains of thought (CoT) [3] and scaling test-time computation [4–7]. Beyond improving final-answer accuracy, these reasoning chains are often surfaced to users as explanations for final answers [8–10] or agent actions [11–15], serving as an important basis for interpretability and user trust. However, long CoTs also introduce substantial inefficiency: models often continue reasoning after a solution has stabilized, repeatedly re-verifying or rephrasing established conclusions [16, 17]. To quantify this redundancy, we analyze five representative LRMs and find that 41–52% of reasoning tokens are generated after the model has already reached its final answer (see Figure 5 in Appendix A). This creates a clear opportunity for more efficient reasoning,

*Equal contribution.

†Corresponding author: dmin10@uic.edu

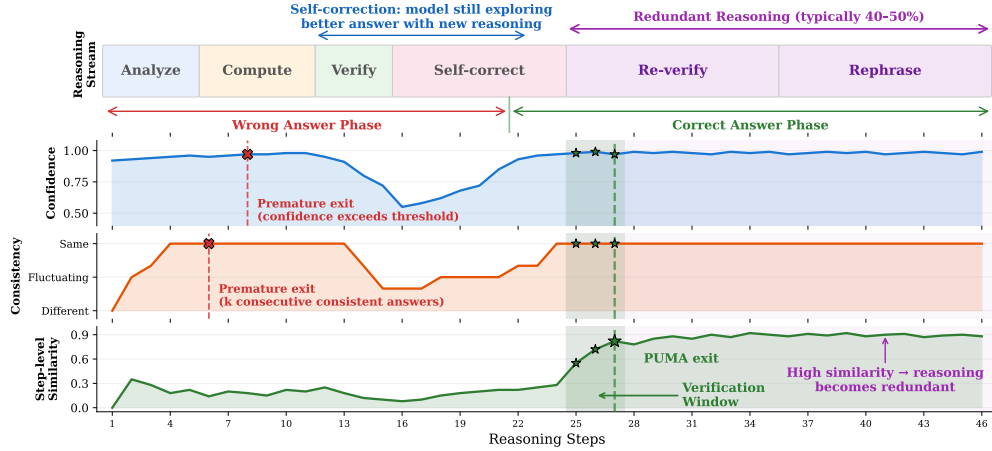


Figure 1: Answer readiness does not imply reasoning convergence. Confidence and trial-answer consistency can trigger premature exits while the model is still exploring or self-correcting. In contrast, step-level semantic similarity remains low during exploration and rises near reasoning convergence, enabling a more reliable exit signal aligned with reasoning convergence.

but the dual role of CoT makes naive truncation inadequate: an effective method should reduce unnecessary tokens while preserving final-answer accuracy and the coherent, semantically complete retained reasoning chain.

A growing body of work has sought to improve reasoning efficiency in LRMs. Training-based methods provide direct control over reasoning length [18–23], but typically require per-model retraining. Prompt-based compression methods are lightweight and easy to apply [24–27], but may hurt accuracy when brevity instructions suppress necessary intermediate reasoning. In contrast, inference-time early-exit methods [28–33] are attractive because they can reduce unnecessary reasoning at deployment time without modifying model weights. However, many existing early-exit methods primarily rely on answer-level signals, such as confidence estimates [32, 34, 28] or trial-answer consistency [35, 33, 30]. While these signals are useful for judging whether the current answer appears stable, they do not directly measure whether the reasoning process has converged. As Figure 1 illustrates, confidence and trial-answer consistency can satisfy stopping criteria while the model is still exploring or self-correcting, leading to premature exits that may degrade final-answer accuracy or truncate important intermediate reasoning. This gap motivates a complementary stopping signal that tracks whether the reasoning trajectory is still making semantically novel progress.

We draw inspiration from semantic entropy [36, 37], which estimates uncertainty in LLM outputs by measuring whether multiple generated responses are semantically diverse or collapse to the same meaning, rather than comparing surface forms. We transfer this idea from *across-output* diversity to *within-trajectory* progress: if recent reasoning steps become semantically similar to prior steps and no longer introduce new logical or semantic content, the reasoning process is likely shifting from exploration to convergence. Under this view, once recent steps become semantically redundant, continued generation is likely to repeat established reasoning rather than add meaningful progress. We therefore use reasoning-level semantic redundancy as a complementary candidate-exit signal.

Building on this signal, we propose **PUMA**, a *Progress-aware Unified Monitoring framework for Adaptive early exit* in reasoning models. PUMA reduces redundant reasoning tokens while preserving both final-answer accuracy and the semantic quality of the retained CoT. It pairs a lightweight Redundancy Detector with answer-level verification: the detector monitors the reasoning trajectory and flags candidate exit points when the current step appears semantically redundant with recent context, while verification checks whether the trial answer is stable and sufficiently confident before stopping. This design decouples *where* to consider stopping from *whether* the candidate exit is reliable, allowing PUMA to avoid relying on answer-level readiness alone. To instantiate the detector, we fine-tune Qwen3-Embedding-0.6B [38] with a contrastive objective [39], training it to distinguish steps that introduce new logical or semantic progress from those that merely restate, re-derive, or loop over prior content. This design also makes PUMA naturally semantic-preserving: because it favors stopping after meaningful exploration, the retained chain is more likely to form a coherent problem-solving narrative rather than cut mid-exploration.

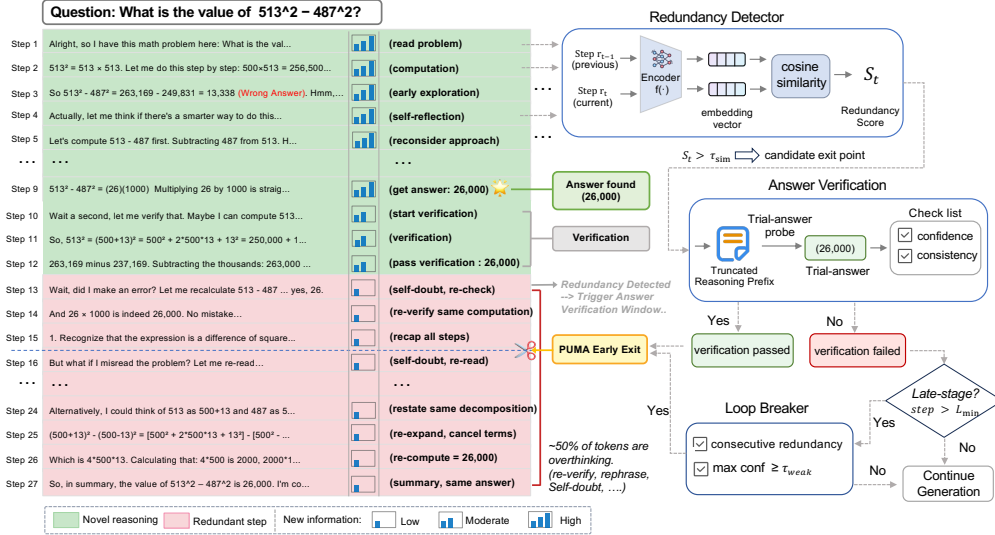


Figure 2: Overview of PUMA. The Redundancy Detector compares recent step embeddings and flags candidate exits. Answer Verification then probes a trial answer from the truncated prefix, checking confidence and consistency before stopping. If verification fails, generation continues; if late-stage consecutive redundancy persists, the Loop Breaker provides a fallback exit. The left trace illustrates PUMA removing redundant continuation while preserving the reasoning prefix.

Across five LRMs from diverse model families, including DeepSeek-R1-Distill [1], Llama-Nemotron [40], and Qwen3 [6], and five challenging reasoning benchmarks spanning MATH-500 [41], AIME24/25 [42, 43], OlympiadBench [44], and GPQA-Diamond [45], PUMA achieves 26.2% average token reduction while preserving final-answer accuracy, with token savings translating into practical wall-clock speedups. Importantly, PUMA also preserves retained CoT quality, indicating that its efficiency gains do not come at the cost of a coherent, semantically complete reasoning chain. Additional experiments on code generation, zero-shot vision-language reasoning, and models trained to internalize PUMA-selected exit positions further show that reasoning-level semantic redundancy is a robust, transferable, and learnable signal for efficient reasoning.

2 Related Work

Overthinking and efficient reasoning in LRMs. Overthinking refers to the phenomenon where extended CoT reasoning, despite improving performance, often generates more tokens than necessary [16, 46–49]. Wei et al. [50] characterize this behavior as a transition from active reasoning to a converged phase in which later tokens are largely redundant. Existing efficient reasoning methods intervene at different stages. **Training-based methods** reshape reasoning through length-penalized RL [51–53], compressed-chain distillation [18, 54], or latent-space reasoning [55, 56], but require per-model retraining [57–59]. **Prompt-based methods** encourage concise reasoning through length budgets or complexity-aware allocation [24, 60, 25], but such constraints can be ignored on difficult problems or suppress necessary intermediate reasoning [61]. PUMA instead focuses on inference-time early exit, reducing redundant continuation without modifying model weights or prompts.

Inference-time early exit. Inference-time early-exit methods differ mainly in the signal used to decide when to stop. *Answer-level signals* monitor trial-answer confidence [32, 34] or agreement across consecutive probes [35, 33, 30]. These signals estimate answer readiness but are blind to whether the reasoning trajectory is still making semantic progress [62–64]. *Token-level signals* track decoding artifacts such as the rank of the $\langle \text{think} \rangle$ token, exit-associated neurons, or reflection-trigger words [50, 65, 66], while *representation-level signals* train hidden-state probes to predict answer correctness [67, 68]. These approaches are often tied to model-specific delimiters, vocabularies, hidden states, or calibration procedures.

3 Methodology

Preliminaries Given an input question, a reasoning model generates a reasoning chain $R = (r_1, r_2, \dots, r_T)$ followed by a final answer A , where each r_t denotes a segmented reasoning step. Step segmentation is performed at natural paragraph or step boundaries, with details in Appendix B.1. PUMA operates online over this step sequence, using reasoning-level redundancy to identify candidate exit points where the trajectory appears to have begun converging, i.e., where recent steps mostly revisit established conclusions rather than add meaningful logical or semantic progress. Because reasoning convergence alone does not imply answer readiness, PUMA exits after answer-level verification checks that the trial answer is stable and sufficiently confident. The goal is to reduce unnecessary tokens while preserving final-answer correctness and a coherent reasoning prefix.

Overview of PUMA Figure 2 illustrates PUMA. PUMA is a plug-and-play early-exit framework that operates during decoding without modifying model weights or the base prompt. It consists of a primary *Verified Early Exit* path and a *Loop Breaker* fallback. Verified Early Exit follows a two-stage design: the Redundancy Detector monitors the reasoning trajectory and flags candidate exit points when the current step appears semantically redundant with recent context; Answer Verification is invoked only at these detector-flagged candidates to check whether the trial answer is stable and sufficiently confident. This separates *where* to consider stopping from *whether* the candidate exit is reliable. The Loop Breaker is activated only when no verified exit occurs and the model produces consecutive redundant steps in the later stages of reasoning.

Redundancy Detector PUMA implements the Redundancy Detector as a lightweight embedding model trained for reasoning-step redundancy detection. We initialize it from Qwen3-Embedding-0.6B [38] and fine-tune it with LoRA [69] using an InfoNCE contrastive objective [39, 70] to distinguish steps that introduce new logical or semantic progress from those that restate, re-derive, or loop over prior content. This task-specific training is important because generic semantic similarity does not directly capture reasoning-step novelty: two steps may be topically similar while still advancing the solution, or lexically different while repeating the same verification. Details on supervision construction, labeling prompts, and training hyperparameters are provided in Appendix B.2.

Given the trained detector $f(\cdot)$, PUMA scores the redundancy of step r_t by comparing it with the previous k reasoning steps:

$$s_t^{(k)} = \max_{\max(1, t-k) \leq j < t} \cos(f(r_j), f(r_t)). \quad (1)$$

A higher score indicates that the current step is semantically close to recent context and is less likely to add novel reasoning progress. PUMA flags r_t as a candidate exit when $s_t^{(k)} > \tau_{\text{sim}}$. By default, we use $k=1$, comparing each step only with its immediate predecessor, which provides a conservative local redundancy signal. Sensitivity to larger lookback windows is analyzed in Appendix B.4; comparisons between our embedding-based detector and Natural Language Inference (NLI)-based [71] redundancy signals are provided in Appendix B.3. Conceptually, this detector serves as a lightweight online proxy for the local semantic collapse that semantic entropy would capture through explicit clustering; we discuss this perspective in Appendix B.5.

Answer Verification. A detector flag is not itself a stopping decision. Once a candidate point t is flagged, PUMA appends a task-specific answer-inducing suffix to the current reasoning prefix $r_{\leq t}$ and probes the model to produce a trial answer A_t . The associated confidence C_t is computed as the geometric mean of token probabilities [32] over the generated answer tokens:

$$C_t = \exp\left(\frac{1}{n} \sum_{i=1}^n \log p(a_i^t | a_{< i}^t, \mathbf{c}_t)\right), \quad (2)$$

where $\mathbf{c}_t$ denotes the prefix plus the answer-inducing suffix. Given the first detector-flagged candidate t_1 , PUMA continues generation until it observes $L-1$ additional detector-flagged candidates, forming a verification window $t_1 < t_2 < \dots < t_L$. At each candidate t_ℓ , PUMA extracts a trial answer A_{t_ℓ} and confidence C_{t_ℓ} . PUMA exits at the end of the window only if

$$\text{Exit}(t_1, \dots, t_L) = [C_{t_1} > \lambda] \wedge \left[\bigwedge_{\ell=2}^L A_{t_\ell} = A_{t_1} \right] \wedge \left[\bigwedge_{\ell=2}^L C_{t_\ell} \geq C_{t_1} - \epsilon \right], \quad (3)$$

where λ is the confidence threshold and ϵ is the stability tolerance. These checks require the candidate answer to be confident, consistent across redundancy-triggered probes, and not materially declining in confidence. If any condition fails, PUMA resumes generation until the next flagged candidate.

Loop Breaker. Verified Early Exit is the primary stopping mechanism, but some trajectories produce consecutive redundant steps without satisfying all verification conditions. PUMA therefore includes a late-stage Loop Breaker fallback. After the reasoning chain exceeds a minimum step threshold, if the Redundancy Detector identifies m consecutive redundant steps, PUMA checks whether the highest-confidence trial answer observed so far exceeds a weak minimum-confidence gate. If the gate is satisfied, PUMA terminates generation; otherwise, generation continues. Verified exits take precedence, so the Loop Breaker is used only when no verified exit occurs.

4 Experimental Setup

Benchmarks and models. We evaluate PUMA on five challenging reasoning benchmarks covering competition mathematics, olympiad-level STEM reasoning, and graduate-level scientific reasoning: MATH-500 [41], AIME24/25 [42, 43], OlympiadBench [44], and GPQA-Diamond [45]. Our main experiments use five LRMs from diverse model families and scales: DeepSeek-R1-Distill-Qwen-7B/14B/32B [1], Llama-3.1-Nemotron-Nano-8B [40], and Qwen3-30B-A3B-Thinking [6]. This suite covers 7B–32B-scale models, dense and mixture-of-experts architectures, and both mathematical and scientific reasoning tasks. Dataset statistics are provided in Appendix C.1.

Baselines. We focus our main comparisons on deployment-compatible efficiency methods that do not modify model weights or require training model-specific hidden-state probes, and report **Full-CoT** as the unmodified generation reference. *Prompt-based:* No-Think [72] prompts the model to skip reasoning entirely; Concise CoT (CCoT; [26]) imposes a global word budget; Chain of Draft (CoD; [24]) enforces per-step word limits; Plan-and-Budget [25] decomposes problems and allocates reasoning budgets by complexity. *Inference-time early exit:* Answer Convergence (Ans. Conv.; [35]) stops when consecutive trial answers agree; Dynasor [33] combines answer consistency with certainty probing; DEER [32] halts when a trial answer exceeds a confidence threshold. All baselines are reproduced using their official code repositories where available, or following the procedures described in the original papers.

Metrics. For the main experiments, we report accuracy (Acc), average generated tokens (Tok), and token reduction $TR = (1 - Tok_{\text{method}}/Tok_{\text{Full-CoT}}) \times 100$. Higher TR indicates greater efficiency. Overall results follow the benchmark-level reporting convention for accuracy, while token-reduction statistics are weighted by benchmark size to reflect aggregate token savings. We report accuracy and token reduction jointly, since token savings are meaningful only when answer quality is preserved. For latency experiments, we additionally report wall-clock speedup relative to Full-CoT. Unless otherwise stated, token counts include all generated tokens incurred by a method, including main reasoning tokens, trial-answer tokens, and final-answer tokens, so TR reflects total generation cost rather than only the retained output length.

Implementation Details. All experiments are conducted on a single node with $4 \times$ NVIDIA GH200 Grace Hopper superchips. We use vLLM for language-model inference and serve the Redundancy Detector on the same node. Unless otherwise specified, we use the models’ recommended reasoning settings (temperature=0.6, top_p =0.95) and report results averaged over three random seeds (0, 42, 123). PUMA uses conservative stopping hyperparameters to favor high-precision exits: for the Redundancy Detector, local window size $k = 1$ and similarity threshold $\tau_{\text{sim}} = 0.35$, selected on a held-out detector calibration split (Appendix B.2); for Answer Verification, confidence threshold $\lambda = 0.98$, stability tolerance $\epsilon = 0.03$, and verification window length $L = 2$; and for the Loop Breaker, a late-stage activation threshold of 50 reasoning steps and a weak minimum-confidence gate of 0.8. Sensitivity to these choices is analyzed in Appendix B.7.

5 Experimental Results

5.1 Main Results: Accuracy, Efficiency, and Reasoning Quality

Table 1 summarizes the accuracy–efficiency tradeoff on three representative LRMs, with full results across all five LRMs reported in Appendix C. Across five LRMs and five benchmarks, PUMA

Table 1: Performance comparison of PUMA against baselines on five benchmarks across three representative reasoning models. Acc (% , $\uparrow$): accuracy. TR (% , $\uparrow$): token reduction relative to Full CoT; negative TR values (shown in **red**) indicate higher token usage than Full CoT. Full results for all five models, including per-benchmark token counts, are reported in Table 14.

Method	MATH-500		AIME24		AIME25		GPQA-D		OlymBench		Overall	
	Acc $\uparrow$	TR $\uparrow$	Acc $\uparrow$	TR $\uparrow$	Acc $\uparrow$	TR $\uparrow$	Acc $\uparrow$	TR $\uparrow$	Acc $\uparrow$	TR $\uparrow$	Acc $\uparrow$	TR $\uparrow$
DeepSeek-R1-Distill-Qwen-7B												
Full CoT	90.0	0.0	50.0	0.0	43.3	0.0	49.0	0.0	57.6	0.0	58.0	0.0
No-Think	79.0	80.0	20.0	67.7	23.3	57.9	28.3	87.3	42.8	79.1	38.7	79.9
CCoT	85.0	60.9	36.7	59.6	33.3	60.2	48.0	55.6	49.2	57.7	50.4	58.6
CoD	80.4	59.9	40.0	56.9	33.3	58.7	50.5	50.8	43.0	55.8	49.4	56.6
Plan&Budget	84.6	48.5	43.3	52.9	23.3	53.9	38.4	36.2	49.3	50.3	47.8	47.9
Ans. Conv.	61.8	76.5	26.7	83.3	20.0	82.8	34.3	92.4	32.6	82.8	35.1	81.9
Dynasor	84.2	30.6	23.3	21.7	33.3	9.1	39.4	66.6	49.0	34.1	45.9	36.6
DEER	90.6	39.6	40.0	14.1	50.0	13.2	51.5	-17.4	57.5	20.2	57.9	21.5
PUMA (ours)	89.6	38.6	60.0	30.3	46.7	29.8	49.0	4.0	55.6	43.2	60.2	35.6
Llama-3.1-Nemotron-Nano-8B												
Full CoT	93.6	0.0	66.7	0.0	50.0	0.0	48.0	0.0	63.1	0.0	64.3	0.0
No-Think	62.8	-125.5	26.7	-32.3	16.7	-43.4	25.8	-2.2	31.9	-74.0	32.8	-80.5
CCoT	86.6	34.5	36.7	36.0	20.0	36.1	43.9	31.1	56.9	34.6	48.8	34.1
CoD	79.0	30.1	50.0	40.9	33.3	39.4	45.0	31.9	51.3	32.0	51.7	31.7
Plan&Budget	89.0	8.3	43.3	32.9	33.3	31.6	40.9	26.7	53.9	22.7	52.1	18.6
Ans. Conv.	55.8	76.3	6.7	91.9	3.3	90.4	20.2	95.3	23.6	85.1	21.9	83.7
Dynasor	91.4	-9.0	50.0	-1.0	43.3	-14.4	45.0	1.5	61.9	-3.0	58.3	-4.7
DEER	91.8	22.7	53.3	-22.5	50.0	-35.9	49.5	-313.2	61.2	12.4	61.2	-30.7
PUMA (ours)	92.6	15.7	70.0	12.3	50.0	18.1	48.5	9.3	62.2	27.0	64.7	20.1
Qwen3-30B-A3B-Thinking												
Full CoT	94.4	0.0	83.3	0.0	83.3	0.0	72.7	0.0	75.0	0.0	81.7	0.0
No-Think	91.8	51.7	60.0	56.5	50.0	50.0	73.7	-11.5	68.4	56.6	68.8	45.3
CCoT	89.2	54.6	36.7	55.9	20.0	60.3	72.7	45.5	53.0	58.8	54.3	55.5
CoD	82.8	29.9	23.3	52.5	10.0	57.8	65.2	28.8	45.3	50.3	45.3	40.4
Plan&Budget	91.8	51.4	43.3	56.6	36.7	60.5	64.7	37.1	61.3	60.2	59.6	53.9
Ans. Conv.	57.0	87.8	0.0	94.4	0.0	90.6	35.9	95.5	23.1	91.7	23.2	90.9
Dynasor	94.8	-4.3	86.7	12.1	76.7	2.8	67.7	32.2	72.4	-0.6	79.7	3.0
DEER	94.8	34.9	83.3	10.5	80.0	7.3	74.8	-8.2	73.5	23.4	81.3	22.4
PUMA (ours)	94.2	28.8	90.0	21.0	80.0	14.7	75.8	17.8	72.3	31.7	82.5	28.2

achieves **26.2%** average token reduction while preserving accuracy. This indicates that PUMA removes redundant continuation without sacrificing final-answer quality. The slight average accuracy gain over Full CoT is consistent with the overthinking phenomenon: after reaching a correct answer, LRMs may continue re-verifying or revising and occasionally drift to an incorrect conclusion.

Compared with prompt-based compression, PUMA is more accuracy-preserving. Although CCoT, CoD, and Plan-and-Budget often reduce token usage, their brevity constraints can suppress necessary intermediate reasoning, especially on stronger reasoning models. For example, on Qwen3-30B-A3B-Thinking, their accuracies drop to 54.3%, 45.3%, and 59.6%, respectively, compared with 81.7% for Full CoT. Compared with answer-level early-exit baselines, PUMA provides a more reliable accuracy–efficiency tradeoff: Answer Convergence stops aggressively but suffers severe accuracy collapse, while Dynasor and DEER show inconsistent efficiency due to repeated trial-answer probing. In contrast, PUMA invokes answer verification only after reasoning-level redundancy is detected.

Beyond final-answer accuracy, early exit should preserve the retained reasoning chain as a useful explanation. Following the LLM-as-Judge protocol [73–75], Table 2 evaluates retained CoT quality along completeness, coherence, conciseness, and justification. PUMA achieves the highest average retained-chain quality among compared methods, ranking first in coherence, conciseness, and justifi-

Table 2: Reasoning-chain quality evaluated by an LLM-as-Judge (GPT-5.4-thinking). Individual chains are rated on a 10–100 rubric in 10-point increments, and the higher is better.

Metric	Full-CoT	PUMA	CCoT	Plan&Budget	CoD	Dynasor	DEER
Completeness $\uparrow$	67.6	64.8	64.1	62.5	62.0	57.2	62.4
Coherence $\uparrow$	41.5	57.5	<u>51.6</u>	48.6	49.0	46.9	37.9
Conciseness $\uparrow$	15.7	40.2	<u>30.5</u>	27.2	26.1	25.0	19.1
Justification $\uparrow$	51.8	54.8	<u>54.5</u>	51.6	50.3	46.5	46.7
Avg. $\uparrow$	44.1	54.3	<u>50.2</u>	47.5	46.8	43.9	41.5

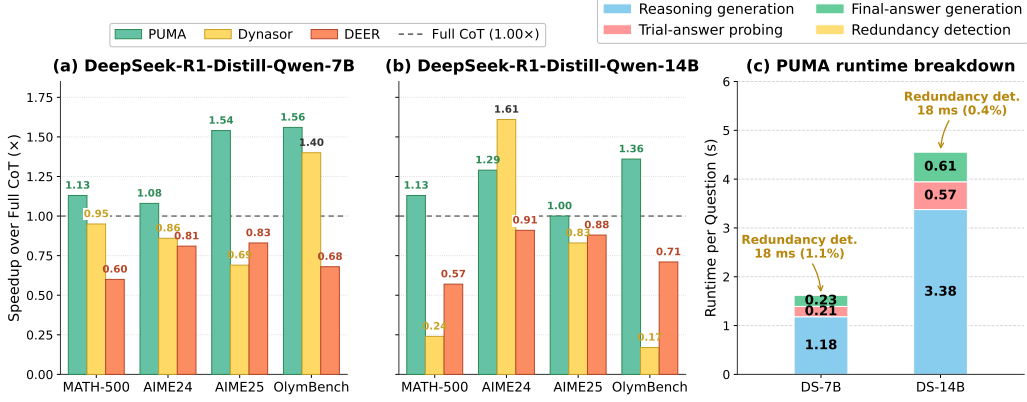


Figure 3: Wall-clock latency on DS-7B and DS-14B. (a,b) Per-benchmark speedup relative to Full-CoT. (c) PUMA runtime breakdown: the Redundancy Detector adds only 0.4–1.1% overhead. Detailed per-benchmark results, including additional models, are reported in Appendix C.5.

cation while remaining close to Full CoT in completeness. This supports the semantic-preserving nature of PUMA: it shortens reasoning by removing redundant continuation rather than aggressively cutting the chain mid-development. Details of the evaluation setup and judge prompt are provided in Appendix E.

5.2 From Token Savings to Latency Gains

Token reduction does not automatically translate to wall-clock speedup, because early-exit methods incur overhead from trial-answer probing. Figure 3 shows that PUMA turns token savings into practical speedups, achieving 1.40× speedup on DS-7B and 1.28× on DS-14B on average. By contrast, DEER is slower than Full CoT on both models, while Dynasor is faster on DS-7B but substantially slower on DS-14B, highlighting the overhead of frequent answer-level probing. The runtime breakdown in Figure 3(c) further shows that PUMA’s overhead is small: the Redundancy Detector contributes only 0.4–1.1% of total wall-clock time per question, and the Answer Verification overhead from trial-answer probing is only 0.2–0.57 seconds per question. PUMA keeps overhead low by using a lightweight detector and probing answers only at detector-flagged candidates.

5.3 Generalization Beyond Text-only QA

Vision-language reasoning introduces unique challenges beyond text-only settings, as models must handle cross-modal interactions [76–78]. We further test whether reasoning-level redundancy transfers beyond text-only mathematical and scientific QA. Table 3 evaluates PUMA on code generation with LiveCodeBench [79] and vision-language reasoning with MathVista [80] and MathVision [81]. On LiveCodeBench, PUMA reduces tokens by 18–19% with at most 1.5 points of pass@1 change. On vision-language reasoning, PUMA is applied without retraining or hyperparameter tuning, yet still reduces tokens by 23.8–33.6% with at most 1.5 points of accuracy change. These results suggest that reasoning-level redundancy is a transferable stopping signal across code, text, and multimodal reasoning. Additional prompt-budget sweeps for CCoT and CoD are provided in Appendix D.

6 Analysis and Discussion

6.1 Component and Exit Behavior Analysis

Table 4 ablates PUMA’s candidate-triggering and verification components on DS-7B, with full two-model results and probe statistics provided in Appendix C.4. The full configuration achieves the best accuracy–efficiency balance, reaching 60.2% accuracy with 35.6% token reduction. Removing the Redundancy Detector gate sets the redundancy threshold to zero, causing Answer Verification to be invoked at every step rather than only at redundancy-triggered candidate points. This aggressive variant increases token reduction to 46.0%, but drops accuracy by 4.1 points and requires 3.3× more trial-answer probes per question. This shows that reasoning-level gating is important for

Table 3: Generalization to code and vision-language reasoning. Code generation is evaluated on LiveCodeBench (880 problems, pass@1); for PUMA, only the redundancy threshold is adjusted to $\tau_{\text{sim}} = 0.50$. Vision-language reasoning is evaluated on MathVista and MathVision (200 problems each), where PUMA is applied zero-shot without retraining or hyperparameter tuning.

Method	Code (LiveCodeBench)				VL: Qwen3-VL-8B-Thinking				VL: Kimi-VL-16B-A3B-Thinking			
	DS-7B		Nemotron-8B		MathVista		MathVision		MathVista		MathVision	
	Acc (Δ)	TR $\uparrow$	Acc (Δ)	TR $\uparrow$	Acc (Δ)	TR $\uparrow$	Acc (Δ)	TR $\uparrow$	Acc (Δ)	TR $\uparrow$	Acc (Δ)	TR $\uparrow$
Full CoT	51.8 (—)	0.0	60.2 (—)	0.0	76.0 (—)	0.0	40.0 (—)	0.0	67.5 (—)	0.0	24.0 (—)	0.0
PUMA (ours)	50.3 (-1.5)	19.0	59.7 (-0.5)	18.0	75.0 (-1.0)	23.8	38.5 (-1.5)	31.2	68.0 (+0.5)	28.7	24.0 (+0.0)	33.6
DEER	49.6 (-2.2)	1.6	52.6 (-7.6)	19.3	71.0 (-5.0)	16.1	36.5 (-3.5)	54.3	53.0 (-14.5)	11.5	22.5 (-1.5)	-1.3
CCoT	45.8 (-6.0)	42.7	49.3 (-10.9)	41.6	72.5 (-3.5)	11.2	36.0 (-4.0)	26.4	53.0 (-14.5)	71.2	21.5 (-2.5)	25.0
CoD	43.2 (-8.6)	40.5	48.9 (-11.3)	41.8	32.5 (-43.5)	60.9	24.5 (-15.5)	58.6	37.5 (-30.0)	38.3	19.5 (-4.5)	42.2
Plan&Budget	39.2 (-12.6)	31.8	50.6 (-9.6)	28.4	62.5 (-13.5)	1.0	30.0 (-10.0)	37.5	57.0 (-10.5)	24.3	22.5 (-1.5)	29.1
Dynasor	43.0 (-8.8)	-207.6	45.5 (-14.7)	-7.8	64.0 (-12.0)	61.8	36.0 (-4.0)	48.4	40.5 (-27.0)	48.3	19.5 (-4.5)	92.6
No-Think	29.1 (-22.7)	88.1	47.6 (-12.6)	13.1	56.0 (-20.0)	63.9	29.5 (-10.5)	73.1	61.5 (-6.0)	70.1	18.5 (-5.5)	68.3

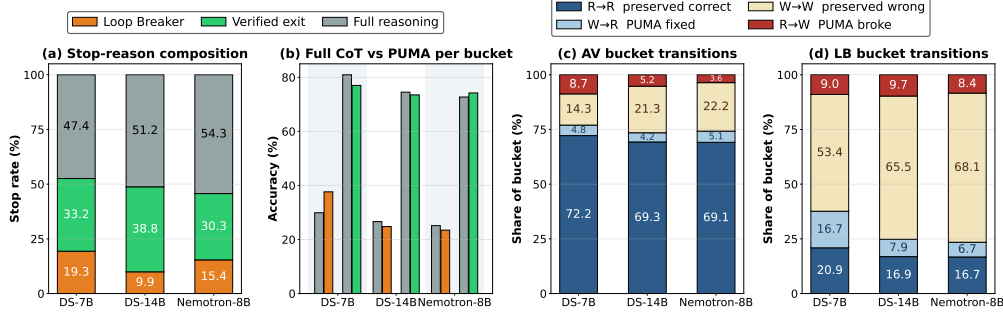


Figure 4: Exit behavior of PUMA across three representative LRMs. (a) Stop-reason composition: Loop Breaker, verified exit, or full reasoning. (b) Accuracy change of each exit bucket relative to Full CoT. (c,d) Correctness transitions for verified exits and Loop Breaker exits, where R/W denote correct/wrong outcomes for Full CoT $\rightarrow$ PUMA.

both reliability and probe efficiency. Removing the Loop Breaker reduces token reduction from 35.6% to 22.6%, confirming that it mainly serves as an efficiency-oriented fallback for sustained redundancy. The Answer-Verification parts are also important for accuracy preservation: removing Answer Consistency lowers accuracy by 2.9 points, while removing the Confidence Gate causes a larger 6.5-point drop despite higher token reduction. Overall, the ablation shows that PUMA’s components are complementary: the Redundancy Detector gate controls when verification is invoked, Answer Verification filters unreliable candidate exits, and the Loop Breaker provides additional savings when reasoning enters sustained redundancy.

Figure 4 further analyzes when PUMA stops and how different exit modes affect correctness. Figure 4(a) shows that PUMA does not rely on a single exit mode: verified exits, Loop Breaker exits, and full reasoning occur in different proportions across models, reflecting model-specific redundancy patterns. Figures 4(c) and 4(d) further break down correctness transitions for verified exits and Loop Breaker exits, respectively, where $W \rightarrow R$, for example, denotes a trajectory that is wrong under Full CoT but correct after PUMA stopping. For verified exits, the $R \rightarrow R$ transition dominates (69–72%), indicating that Answer Verification usually preserves already-correct trajectories. The $W \rightarrow R$ cases show that PUMA can also fix some Full-CoT errors by stopping before post-convergence drift, while harmful $R \rightarrow W$ transitions remain limited. Loop Breaker exits show a different pattern: $W \rightarrow W$ transitions dominate (53–68%), meaning that the fallback mostly cuts late redundant continuation from trajectories that Full CoT would also answer incorrectly. Together, these results explain how PUMA gains efficiency with limited accuracy risk, and why it can occasionally outperform Full CoT by avoiding late-stage drift.

Table 4: Component ablation on DS-7B, averaged over five benchmarks. Probe $\times$ denotes trial-answer probes per question relative to PUMA.

Configuration	Acc $\uparrow$	TR $\uparrow$	Probe $\times$ $\downarrow$
PUMA (full)	60.2	35.6	1.0$\times$
w/o Redundancy Detector Gate	56.1 (-4.1)	46.0	3.3 $\times$
w/o Loop Breaker	59.2 (-1.0)	22.6	1.3 $\times$
w/o Answer Consistency	57.3 (-2.9)	37.8	1.2 $\times$
w/o Confidence Gate	53.7 (-6.5)	55.3	0.6 $\times$

Table 5: Internalizing PUMA’s stopping signal with LoRA on DS-R1-Distill-Qwen-7B. PUMA-selected exit positions supervise SFT, DPO, or GRPO; all trained variants use pure vLLM inference without PUMA modules.

Method	MATH-500		AIME24		GPQA-D		Avg.	
	Acc↑	TR↑	Acc↑	TR↑	Acc↑	TR↑	Acc↑	TR↑
Full CoT (base)	90.0	0.0	50.0	0.0	49.0	0.0	63.0	0.0
PUMA (inference)	89.6	38.6	60.0	30.3	49.0	4.0	66.2	24.3
<i>Imitation learning (SFT)</i>								
Standard-SFT	91.4	-4.5	43.3	-31.0	49.5	-61.9	61.4	-32.5
FixedExit-SFT	85.8	46.3	36.7	17.2	39.9	21.3	54.1	28.3
PUMA-SFT	91.4	21.9	53.3	32.7	56.1	3.3	66.9	19.3
<i>Preference learning (DPO)</i>								
PUMA-DPO	90.6	39.3	56.7	59.4	45.0	47.7	64.1	48.8
<i>Reinforcement learning (GRPO)</i>								
Standard-GRPO	91.4	1.2	50.0	-1.5	56.1	-9.8	65.8	-3.4
FixedExit-RL	91.4	41.5	46.7	38.0	51.5	32.8	63.2	37.4
PUMA-RL	90.2	42.4	56.7	35.7	54.0	26.6	67.0	34.9

6.2 From Inference-Time Signal to Learned Stopping Policy

Finally, we ask whether PUMA-selected exit positions can be internalized into the model itself. We fine-tune DS-R1-Distill-Qwen-7B on 12K math problems using SFT, DPO [82], and GRPO [83], with PUMA-selected exits providing stopping supervision. All trained variants are evaluated with pure vLLM inference and use no PUMA modules at deployment. Each paradigm uses PUMA’s exit positions differently. PUMA-SFT uses PUMA-truncated reasoning traces and their final answers as supervised targets, teaching the model to produce concise reasoning paths that stop near PUMA-selected exit points. PUMA-DPO treats PUMA-truncated chains as preferred over full-length chains when both produce correct answers, teaching the model that shorter-is-better given equal correctness. PUMA-RL uses GRPO on rollouts launched from PUMA’s RD-flagged exit positions, with a reward that combines correctness, a length bonus, and a within-group rank bonus that favors the shortest correct trajectory. Detailed data construction and training settings are provided in Appendix F. Table 5 shows that PUMA-selected exits provide useful supervision for concise reasoning. PUMA-RL achieves the strongest overall result, exceeding training-free PUMA in both average accuracy (67.0 vs. 66.2) and token reduction (34.9% vs. 24.3%). PUMA-DPO provides a more compression-oriented alternative, reaching the highest average token reduction (48.8%) while still improving over the Full CoT baseline in average accuracy (64.1 vs. 63.0). The FixedExit baselines use the same training recipes but replace PUMA-selected positions with fixed-interval candidates. Their weaker performance indicates that PUMA’s exit positions are semantically informative rather than merely shorter prefixes: PUMA-SFT improves over FixedExit-SFT by 12.8 accuracy points on average, and PUMA-RL improves over FixedExit-RL by 3.8 points while maintaining comparable token reduction. In contrast, Standard-SFT and Standard-GRPO do not learn concise stopping behavior, using more tokens than Full CoT on average (i.e., average token reduction ≤ 0). These results suggest that reasoning-level redundancy is not only useful for inference-time early exit, but can also be learned as a stopping policy.

7 Conclusion

We presented PUMA, a plug-and-play early-exit framework that uses reasoning-level semantic redundancy to identify when a reasoning trajectory appears to have converged, and pairs this signal with answer-level verification for reliable early stopping. Across five LRMs and five challenging reasoning benchmarks, PUMA reduces tokens by 26.2% on average while preserving accuracy, delivering practical wall-clock speedups, and maintaining higher-quality retained reasoning chains than competing methods. PUMA further transfers to code generation and zero-shot vision-language reasoning, and its selected exit positions can supervise SFT, DPO, and GRPO, enabling models to internalize concise stopping behavior without PUMA modules at inference time. These results suggest that reasoning-level redundancy is a robust, transferable, and learnable signal for efficient reasoning.

Limitations. PUMA relies on step-level reasoning traces and may be less effective when model outputs are very short, poorly structured, or difficult to segment. Although the Redundancy Detector transfers beyond text-only QA, it is primarily trained on text reasoning traces, so broader domains

may require additional calibration. Our internalization experiments are also limited to one base model and math-focused training data, leaving larger-scale and cross-domain learned stopping policies for future work.

References

- [1] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638, 2025.
- [2] Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, et al. Openai o1 system card. *arXiv preprint arXiv:2412.16720*, 2024.
- [3] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, brian ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/9d5609613524ecf4f15af0f7b31abca4-Paper-Conference.pdf.
- [4] Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candes, and Tatsunori Hashimoto. s1: Simple test-time scaling. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 20275–20321, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.1025. URL <https://aclanthology.org/2025.emnlp-main.1025/>.
- [5] Wenkai Yang, Shuming Ma, Yankai Lin, and Furu Wei. Towards thinking-optimal scaling of test-time compute for LLM reasoning. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL <https://openreview.net/forum?id=6ICFqmixlS>.
- [6] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [7] Charlie Victor Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling parameters for reasoning. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [8] Tomek Korbak, Mikita Balesni, Elizabeth Barnes, Yoshua Bengio, Joe Benton, Joseph Bloom, Mark Chen, Alan Cooney, Allan Dafoe, Anca Dragan, et al. Chain of thought monitorability: A new and fragile opportunity for ai safety. *arXiv preprint arXiv:2507.11473*, 2025.
- [9] Jiaxuan Gao, Shu Yan, Qixin Tan, lu Yang, Shusheng Xu, Wei Fu, Zhiyu Mei, Kaifeng Lyu, and Yi Wu. How far are we from optimal reasoning efficiency? In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL <https://openreview.net/forum?id=NhA1w3s8Z>.
- [10] Zhong-Zhi Li, Xiao Liang, Zihao Tang, Lei Ji, Peijie Wang, Haotian Xu, Xing W, Haizhen Huang, Weiwei Deng, Yeyun Gong, Zhijiang Guo, Xiao Liu, Fei Yin, and Cheng-Lin Liu. Tl;dr: Too long, do re-weighting for efficient llm reasoning compression, 2025. URL <https://arxiv.org/abs/2506.02678>.
- [11] Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language agent tree search unifies reasoning, acting, and planning in language models. In *Proceedings of the 41st International Conference on Machine Learning, ICML’24*. JMLR.org, 2024.

- [12] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.
- [13] Chan-Jan Hsu, Davide Buffelli, Jamie McGowan, Feng-Ting Liao, Yi-Chang Chen, Sattar Vakili, and Da shan Shiu. Group think: Multiple concurrent reasoning agents collaborating at token level granularity, 2025. URL <https://arxiv.org/abs/2505.11107>.
- [14] Yu Li, Guangfeng Cai, Shengtian Yang, Han Luo, Shuo Han, Xu He, Dong Li, and Lei Feng. Phgpo: Pheromone-guided policy optimization for long-horizon tool planning. *arXiv preprint arXiv:2602.13691*, 2026.
- [15] Henry Peng Zou, Wei-Chieh Huang, Yaozu Wu, Yankai Chen, Chunyu Miao, Hoang Nguyen, Yue Zhou, Weizhi Zhang, Liancheng Fang, Langzhou He, et al. Llm-based human-agent collaboration and interaction systems: A survey. *arXiv preprint arXiv:2505.00753*, 2025.
- [16] Xingyu Chen, Jiahao Xu, Tian Liang, Zhiwei He, Jianhui Pang, Dian Yu, Linfeng Song, Qiuzhi Liu, Mengfei Zhou, Zhuosheng Zhang, Rui Wang, Zhaopeng Tu, Haitao Mi, and Dong Yu. Do NOT think that much for $2+3=?$ on the overthinking of long reasoning models. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=MSbU3L7V00>.
- [17] Renliang Sun, Wei Cheng, Dawei Li, Haifeng Chen, and Wei Wang. Stop when enough: Adaptive early-stopping for chain-of-thought reasoning. *arXiv preprint arXiv:2510.10103*, 2025.
- [18] Jintian Zhang, Yuqi Zhu, Mengshu Sun, Yujie Luo, Shuofei Qiao, Lun Du, Da Zheng, Huajun Chen, and Ningyu Zhang. LightThinker: Thinking step-by-step compression. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 13307–13328, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.673. URL <https://aclanthology.org/2025.emnlp-main.673/>.
- [19] Daman Arora and Andrea Zanette. Training language models to reason efficiently. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=AiZxn84Wdo>.
- [20] Haotian Luo, Li Shen, Haiying He, Yibo Wang, Shiwei Liu, Wei Li, Naiqiang Tan, Xiaochun Cao, and Dacheng Tao. O1-pruner: Length-harmonizing fine-tuning for o1-like reasoning pruning. In *2nd AI for Math Workshop @ ICML 2025*, 2025. URL <https://openreview.net/forum?id=ioYybCRcyW>.
- [21] Yu Kang, Xianghui Sun, Liangyu Chen, and Wei Zou. C3ot: Generating shorter chain-of-thought without compromising effectiveness. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 24312–24320, 2025.
- [22] Xinyin Ma, Guangnian Wan, Runpeng Yu, Gongfan Fang, and Xinchao Wang. CoT-valve: Length-compressible chain-of-thought tuning. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6025–6035, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.300. URL <https://aclanthology.org/2025.acl-long.300/>.
- [23] Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, et al. Kimi k1. 5: Scaling reinforcement learning with llms. *arXiv preprint arXiv:2501.12599*, 2025.
- [24] Silei Xu, Wenhao Xie, Lingxiao Zhao, and Pengcheng He. Chain of draft: Thinking faster by writing less. *arXiv preprint arXiv:2502.18600*, 2025.

- [25] Junhong Lin, Xinyue Zeng, Jie Zhu, Song Wang, Julian Shun, Jun Wu, and Dawei Zhou. Plan and budget: Effective and efficient test-time scaling on reasoning large language models. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=ctspw4CqbS>.
- [26] Sania Nayab, Giulio Rossolini, Marco Simoni, Andrea Saracino, Giorgio Buttazzo, Nicolamaria Manes, and Fabrizio Giacomelli. Concise thoughts: Impact of output length on llm reasoning and cost. *arXiv preprint arXiv:2407.19825*, 2024.
- [27] Tingxu Han, Zhenting Wang, Chunrong Fang, Shiyu Zhao, Shiqing Ma, and Zhenyu Chen. Token-budget-aware LLM reasoning. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Findings of the Association for Computational Linguistics: ACL 2025*, pages 24842–24855, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-256-5. doi: 10.18653/v1/2025.findings-acl.1274. URL <https://aclanthology.org/2025.findings-acl.1274/>.
- [28] Aaroosh Rustagi, Hsien Xin Peng, Khushal Murthy, Attrey Koul, Ryan Lagasse, and Kevin Zhu. Confidence-coverage gating for early exit. In *NeurIPS 2025 Workshop on Efficient Reasoning*, 2025. URL <https://openreview.net/forum?id=Ay7sRmWswq>.
- [29] Chengsong Huang, Langlin Huang, Jixuan Leng, Jiacheng Liu, and Jiaxin Huang. Efficient test-time scaling via self-calibration. In *NeurIPS 2025 Workshop on Efficient Reasoning*, 2025. URL <https://openreview.net/forum?id=RvMjxGpV0a>.
- [30] Minjia Mao, Bowen Yin, Yu Zhu, and Xiao Fang. Early stopping chain-of-thoughts in large language models. *arXiv preprint arXiv:2509.14004*, 2025.
- [31] Xi Wang, James McInerney, Lequn Wang, and Nathan Kallus. Entropy after $\{\text{/Think}\}$ for reasoning model early exiting. *arXiv preprint arXiv:2509.26522*, 2025.
- [32] Chenxu Yang, Qingyi Si, Yongjie Duan, Zheliang Zhu, Chenyu Zhu, Qiaowei Li, Minghui Chen, Zheng Lin, and Weiping Wang. Dynamic early exit in reasoning models. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=NpU7ZXafRi>.
- [33] Yichao Fu, Junda Chen, Yonghao Zhuang, Zheyu Fu, Ion Stoica, and Hao Zhang. Reasoning without self-doubt: More efficient chain-of-thought through certainty probing. In *ICLR 2025 Workshop on Foundation Models in the Wild*, 2025. URL <https://openreview.net/forum?id=wpK4IMJfdX>.
- [34] Jiameng Huang, Baijiong Lin, Guhao Feng, Jierun Chen, Di He, and Lu Hou. Efficient reasoning for large reasoning language models via certainty-guided reflection suppression. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 31176–31184, 2026.
- [35] Xin Liu and Lu Wang. Answer convergence as a signal for early stopping in reasoning. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 17896–17907, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.904. URL <https://aclanthology.org/2025.emnlp-main.904/>.
- [36] Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=VD-AYtP0dve>.
- [37] Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, and Yarin Gal. Detecting hallucinations in large language models using semantic entropy. *Nature*, 630(8017):625–630, 2024.
- [38] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. Qwen3 embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176*, 2025.

- [39] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018.
- [40] Akhiad Bercovich, Itay Levy, Izik Golan, Mohammad Dabbah, Ran El-Yaniv, Omri Puny, Ido Galil, Zach Moshe, Tomer Ronen, Najeeb Nabwani, Ido Shahaf, Oren Tropp, Ehud Karpas, Ran Zilberstein, Jiaqi Zeng, Soumye Singhal, Alexander Bukharin, Yian Zhang, Tugrul Konuk, Gerald Shen, Ameya Sunil Mahabaleshwarkar, Bilal Kartal, Yoshi Suhara, Olivier Delalleau, Zijia Chen, Zhilin Wang, David Mosallanezhad, Adi Renduchintala, Haifeng Qian, Dima Rekesh, Fei Jia, Somshubra Majumdar, Vahid Noroozi, Wasi Uddin Ahmad, Sean Narenthiran, Aleksander Ficek, Mehrzad Samadi, Jocelyn Huang, Siddhartha Jain, Igor Gitman, Ivan Moshkov, Wei Du, Shubham Toshniwal, George Armstrong, Branislav Kisacanin, Matvei Novikov, Daria Gitman, Evelina Bakhturina, Jane Polak Scowcroft, John Kamalu, Dan Su, Kezhi Kong, Markus Kliegl, Rabeeh Karimi, Ying Lin, Sanjeev Satheesh, Jupinder Parmar, Pritam Gundecha, Brandon Norick, Joseph Jennings, Shrimai Prabhumoye, Syeda Nahida Akter, Mostofa Patwary, Abhinav Khattar, Deepak Narayanan, Roger Waleffe, Jimmy Zhang, Bor-Yiing Su, Guyue Huang, Terry Kong, Parth Chadha, Sahil Jain, Christine Harvey, Elad Segal, Jining Huang, Sergey Kashirsky, Robert McQueen, Izzy Putterman, George Lam, Arun Venkatesan, Sherry Wu, Vinh Nguyen, Manoj Kilaru, Andrew Wang, Anna Warno, Abhilash Somasamudramath, Sandip Bhaskar, Maka Dong, Nave Assaf, Shahar Mor, Omer Ullman Argov, Scot Junkin, Oleksandr Romanenko, Pedro Larroy, Monika Katariya, Marco Rovinelli, Viji Balas, Nicholas Edelman, Anahita Bhiwandiwalla, Muthu Subramaniam, Smita Ithape, Karthik Ramamoorthy, Yuting Wu, Suguna Varshini Velury, Omri Almog, Joyjit Daw, Denys Fridman, Erick Galinkin, Michael Evans, Katherine Luna, Leon Derczynski, Nikki Pope, Eileen Long, Seth Schneider, Guillermo Siman, Tomasz Grzegorzec, Pablo Ribalta, Monika Katariya, Joey Conway, Trisha Saar, Ann Guan, Krzysztof Pawelec, Shyamala Prayaga, Oleksii Kuchaiev, Boris Ginsburg, Oluwatobi Olabiyi, Kari Briski, Jonathan Cohen, Bryan Catanzaro, Jonah Alben, Yonatan Geifman, Eric Chung, and Chris Alexiuk. Llama-nemotron: Efficient reasoning models, 2025. URL <https://arxiv.org/abs/2505.00949>.
- [41] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*, 2021. URL <https://openreview.net/forum?id=7Bywt2mQsCe>.
- [42] Yifan Zhang and Team Math-AI. American invitational mathematics examination (aime) 2024, 2024.
- [43] Yifan Zhang and Team Math-AI. American invitational mathematics examination (aime) 2025, 2025.
- [44] Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, Jie Liu, Lei Qi, Zhiyuan Liu, and Maosong Sun. OlympiadBench: A challenging benchmark for promoting AGI with olympiad-level bilingual multimodal scientific problems. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3828–3850, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.211. URL <https://aclanthology.org/2024.acl-long.211/>.
- [45] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. GPQA: A graduate-level google-proof q&a benchmark. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=Ti67584b98>.
- [46] Yang Sui, Yu-Neng Chuang, Guanchu Wang, Jiamu Zhang, Tianyi Zhang, Jiayi Yuan, Hongyi Liu, Andrew Wen, Shaochen Zhong, Na Zou, Hanjie Chen, and Xia Hu. Stop overthinking: A survey on efficient reasoning for large language models. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856. URL <https://openreview.net/forum?id=HvoG8SxggZ>.
- [47] Jinyan Su, Jennifer Healey, Preslav Nakov, and Claire Cardie. Between underthinking and overthinking: An empirical study of reasoning length and correctness in llms. *arXiv preprint arXiv:2505.00127*, 2025.

- [48] Soumya Suvra Ghosal, Souradip Chakraborty, Avinash Reddy, Yifu Lu, Mengdi Wang, Dinesh Manocha, Furong Huang, Mohammad Ghavamzadeh, and Amrit Singh Bedi. Does thinking more always help? mirage of test-time scaling in reasoning models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL <https://openreview.net/forum?id=tKPqbamNb9>.
- [49] Yuyang Wu, Yifei Wang, Tianqi Du, Stefanie Jegelka, and Yisen Wang. When more is less: Understanding chain-of-thought length in LLMs. In *Workshop on Reasoning and Planning for Large Language Models*, 2025. URL <https://openreview.net/forum?id=W8dxn7hBk0>.
- [50] Zihao Wei, Liang Pang, Jiahao Liu, Wenjie Shi, Jingcheng Deng, Shicheng Xu, Zenghao Duan, Fei Sun, Huawei Shen, and Xueqi Cheng. The evolution of thought: Tracking llm overthinking via reasoning dynamics analysis, 2026. URL <https://arxiv.org/abs/2508.17627>.
- [51] Pranjal Aggarwal and Sean Welleck. L1: Controlling how long a reasoning model thinks with reinforcement learning. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=4jdIxXBnve>.
- [52] Bairu Hou, Yang Zhang, Jiabao Ji, Yujian Liu, Kaizhi Qian, Jacob Andreas, and Shiyu Chang. Thinkprune: Pruning long chain-of-thought of LLMs via reinforcement learning. *Transactions on Machine Learning Research*, 2026. ISSN 2835-8856. URL <https://openreview.net/forum?id=V51gPu1uQD>.
- [53] Mz Dai, Chenxu Yang, and Qingyi Si. S-GRPO: Early exit via reinforcement learning in reasoning models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL <https://openreview.net/forum?id=wNMK5o0Vfg>.
- [54] Zeju Li, Jianyuan Zhong, Ziyang Zheng, Xiangyu Wen, Zhijian Xu, Yingying Cheng, Fan Zhang, and Qiang Xu. Making slow thinking faster: Compressing LLM chain-of-thought via step entropy. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=cGLqQfS5wH>.
- [55] Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason E Weston, and Yuandong Tian. Training large language models to reason in a continuous latent space. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=Itxz7S4Ip3>.
- [56] Yinhan He, Wendy Zheng, Yaochen Zhu, Zaiyi Zheng, Lin Su, Sriram Vasudevan, Qi Guo, Liangjie Hong, and Jundong Li. Semcot: Accelerating chain-of-thought reasoning through semantically-aligned implicit tokens. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=1ZuzFUMtx6>.
- [57] Yixin Ye, Zhen Huang, Yang Xiao, Ethan Chern, Shijie Xia, and Pengfei Liu. LIMO: Less is more for reasoning. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=T2TZ0RY4Zk>.
- [58] Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, YuYue, Weinan Dai, Tiantian Fan, Gaohong Liu, Juncai Liu, LingJun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Ru Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Yonghui Wu, and Mingxuan Wang. DAPO: An open-source LLM reinforcement learning system at scale. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL <https://openreview.net/forum?id=2a36EMSSTp>.
- [59] Kianté Brantley, Mingyu Chen, Zhaolin Gao, Jason D. Lee, Wen Sun, Wenhao Zhan, and Xuezhou Zhang. Accelerating RL for LLM reasoning with optimal advantage regression. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL <https://openreview.net/forum?id=T1V8BJ00iG>.
- [60] Matthew Renze and Erhan Guven. The benefits of a concise chain of thought on problem-solving in large language models. In *2024 2nd International Conference on Foundation and Large Language Models (FLLM)*, pages 476–483, 2024. doi: 10.1109/FLLM63129.2024.10852493.

- [61] Ye Yu, Yaoning Yu, Haibo Jin, and Haohan Wang. PREMISE: Scalable and strategic prompt optimization for efficient mathematical reasoning in large reasoning models. In *NeurIPS 2025 Workshop on Efficient Reasoning*, 2025. URL <https://openreview.net/forum?id=8mI3i9LXj3>.
- [62] Katherine Tian, Eric Mitchell, Allan Zhou, Archit Sharma, Rafael Rafailov, Huaxiu Yao, Chelsea Finn, and Christopher D Manning. Just ask for calibration: Strategies for eliciting calibrated confidence scores from language models fine-tuned with human feedback. In *The 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. URL <https://openreview.net/forum?id=g3faCfrwm7>.
- [63] Prateek Chhikara. Mind the confidence gap: Overconfidence, calibration, and distractor effects in large language models. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856. URL <https://openreview.net/forum?id=1yaHnHDdZl>.
- [64] Adam Tauman Kalai and Santosh S. Vempala. Calibrated language models must hallucinate. In *Proceedings of the 56th Annual ACM Symposium on Theory of Computing*, STOC 2024, page 160–171, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400703836. doi: 10.1145/3618260.3649777. URL <https://doi.org/10.1145/3618260.3649777>.
- [65] Kang Liu, Yongkang Liu, Xiaocui Yang, Peidong Wang, Wen Zhang, Shi Feng, Yifei Zhang, and Daling Wang. Neat: Neuron-based early exit for large reasoning models. *arXiv preprint arXiv:2602.02010*, 2026.
- [66] Chenlong Wang, Yuanning Feng, Dongping Chen, Zhaoyang Chu, Ranjay Krishna, and Tianyi Zhou. Wait, we don’t need to “wait”! removing thinking tokens improves reasoning efficiency. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 7459–7482, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-335-7. doi: 10.18653/v1/2025.findings-emnlp.394. URL <https://aclanthology.org/2025.findings-emnlp.394/>.
- [67] Ömer Faruk Akgül, Yusuf Hakan Kalaycı, Rajgopal Kannan, Willie Neiswanger, and Viktor Prasanna. Lynx: Learning dynamic exits for confidence-controlled reasoning. *arXiv preprint arXiv:2512.05325*, 2025.
- [68] Anqi Zhang, Yulin Chen, Jane Pan, Chen Zhao, Aurojit Panda, Jinyang Li, and He He. Reasoning models know when they’re right: Probing hidden states for self-verification. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=06IOAv7683>.
- [69] Edward J Hu, yelong shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuezhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=nZeVKeeFYf9>.
- [70] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In Marina Meila and Tong Zhang, editors, *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 8748–8763. PMLR, 18–24 Jul 2021. URL <https://proceedings.mlr.press/v139/radford21a.html>.
- [71] Bill MacCartney. *Natural language inference*. Stanford University, 2009.
- [72] Wenjie Ma, Jingxuan He, Charlie Snell, Tyler Griggs, Sewon Min, and Matei Zaharia. Reasoning models can be effective without thinking. *arXiv preprint arXiv:2504.09858*, 2025.
- [73] Dawei Li, Bohan Jiang, Liangjie Huang, Alimohammad Beigi, Chengshuai Zhao, Zhen Tan, Amrita Bhattacharjee, Yuxuan Jiang, Canyu Chen, Tianhao Wu, Kai Shu, Lu Cheng, and Huan Liu. From generation to judgment: Opportunities and challenges of LLM-as-a-judge. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Proceedings*

- of the 2025 Conference on Empirical Methods in Natural Language Processing, pages 2757–2791, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.138. URL <https://aclanthology.org/2025.emnlp-main.138/>.
- [74] Yu Li, Shenyu Zhang, Rui Wu, Xiutian Huang, Yongrui Chen, Wenhao Xu, Guilin Qi, and Dehai Min. Mateval: a multi-agent discussion framework for advancing open-ended text evaluation. In *International Conference on Database Systems for Advanced Applications*, pages 415–426. Springer, 2024.
 - [75] Shenyu Zhang, Yu Li, Rui Wu, Xiutian Huang, Yongrui Chen, Wenhao Xu, and Guilin Qi. Dee: Dual-stage explainable evaluation method for text generation. In *International Conference on Database Systems for Advanced Applications*, pages 390–401. Springer, 2024.
 - [76] Li Li, Jiawei Peng, Huiyi Chen, Chongyang Gao, and Xu Yang. How to configure good in-context sequence for visual question answering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 26710–26720, June 2024.
 - [77] Huiyi Chen, Jiawei Peng, Dehai Min, Changchang Sun, Kaijie Chen, Yan Yan, Xu Yang, and Lu Cheng. Mvi-bench: A comprehensive benchmark for evaluating robustness to misleading visual inputs in lvlms. *arXiv preprint arXiv:2511.14159*, 2025.
 - [78] Huiyi Chen, Jiawei Peng, Kaihua Tang, Xin Geng, and Xu Yang. Enhancing multimodal in-context learning for image classification through coreset optimization. In *Proceedings of the 33rd ACM International Conference on Multimedia*, pages 5130–5139, 2025.
 - [79] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=chfJJYC3iL>.
 - [80] Pan Lu, Hritik Bansal, Tony Xia, Jiacheng Liu, Chunyuan Li, Hannaneh Hajishirzi, Hao Cheng, Kai-Wei Chang, Michel Galley, and Jianfeng Gao. Mathvista: Evaluating mathematical reasoning of foundation models in visual contexts. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=KUNzEQMWU7>.
 - [81] Ke Wang, Juntong Pan, Weikang Shi, Zimu Lu, Houxing Ren, Aojun Zhou, Mingjie Zhan, and Hongsheng Li. Measuring multimodal mathematical reasoning with math-vision dataset. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2024. URL <https://openreview.net/forum?id=QWTCcxMpPA>.
 - [82] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 53728–53741. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/a85b405ed65c6477a4fe8302b5e06ce7-Paper-Conference.pdf.
 - [83] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
 - [84] Ziqing Qiao, Yongheng Deng, Jiali Zeng, Dong Wang, Lai Wei, Guanbo Wang, Fandong Meng, Jie Zhou, Ju Ren, and Yaoxue Zhang. ConCISE: Confidence-guided compression in step-by-step efficient reasoning. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 8010–8029, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.405. URL <https://aclanthology.org/2025.emnlp-main.405/>.
 - [85] Qwen Team. Qwq-32b: Embracing the power of reinforcement learning, March 2025. URL <https://qwenlm.github.io/blog/qwq-32b/>.

- [86] OpenAI. gpt-oss-120b & gpt-oss-20b model card, 2025. URL <https://arxiv.org/abs/2508.10925>.
- [87] GLM Team, Aohan Zeng, Xin Lv, Qinkai Zheng, Zhenyu Hou, Bin Chen, Chengxing Xie, Cunxiang Wang, Da Yin, Hao Zeng, Jiajie Zhang, Kedong Wang, Lucen Zhong, Mingdao Liu, Rui Lu, Shulin Cao, Xiaohan Zhang, Xuancheng Huang, Yao Wei, Yean Cheng, Yifan An, Yilin Niu, Yuanhao Wen, Yushi Bai, Zhengxiao Du, Zihan Wang, Zilin Zhu, Bohan Zhang, Bosi Wen, Bowen Wu, Bowen Xu, Can Huang, Casey Zhao, Changpeng Cai, Chao Yu, Chen Li, Chendi Ge, Chenghua Huang, Chenhui Zhang, Chenxi Xu, Chenzheng Zhu, Chuang Li, Congfeng Yin, Daoyan Lin, Dayong Yang, Dazhi Jiang, Ding Ai, Erle Zhu, Fei Wang, Gengzheng Pan, Guo Wang, Hailong Sun, Haitao Li, Haiyang Li, Haiyi Hu, Hanyu Zhang, Hao Peng, Hao Tai, Haoke Zhang, Haoran Wang, Haoyu Yang, He Liu, He Zhao, Hongwei Liu, Hongxi Yan, Huan Liu, Huilong Chen, Ji Li, Jiajing Zhao, Jiamin Ren, Jian Jiao, Jiani Zhao, Jianyang Yan, Jiaqi Wang, Jiayi Gui, Jiayue Zhao, Jie Liu, Jijie Li, Jing Li, Jing Lu, Jingsen Wang, Jingwei Yuan, Jingxuan Li, Jingzhao Du, Jinhua Du, Jinxin Liu, Junkai Zhi, Junli Gao, Ke Wang, Lekang Yang, Liang Xu, Lin Fan, Lindong Wu, Lintao Ding, Lu Wang, Man Zhang, Minghao Li, Minghuan Xu, Mingming Zhao, Mingshu Zhai, Pengfan Du, Qian Dong, Shangde Lei, Shangqing Tu, Shangtong Yang, Shaoyou Lu, Shijie Li, Shuang Li, Shuang-Li, Shuxun Yang, Sibao Yi, Tianshu Yu, Wei Tian, Weihang Wang, Wenbo Yu, Weng Lam Tam, Wenjie Liang, Wentao Liu, Xiao Wang, Xiaohan Jia, Xiaotao Gu, Xiaoying Ling, Xin Wang, Xing Fan, Xingru Pan, Xinyuan Zhang, Xinze Zhang, Xiuqing Fu, Xunkai Zhang, Yabo Xu, Yandong Wu, Yida Lu, Yidong Wang, Yilin Zhou, Yiming Pan, Ying Zhang, Yingli Wang, Yingru Li, Yinpei Su, Yipeng Geng, Yitong Zhu, Yongkun Yang, Yuhang Li, Yuhao Wu, Yujiang Li, Yunan Liu, Yunqing Wang, Yuntao Li, Yuxuan Zhang, Zezhen Liu, Zhen Yang, Zhengda Zhou, Zhongpei Qiao, Zhuoer Feng, Zhuorui Liu, Zichen Zhang, Zihan Wang, Zijun Yao, Zikang Wang, Ziqiang Liu, Ziwei Chai, Zixuan Li, Zuodong Zhao, Wenguang Chen, Jidong Zhai, Bin Xu, Minlie Huang, Hongning Wang, Juanzi Li, Yuxiao Dong, and Jie Tang. Glm-4.5: Agentic, reasoning, and coding (arc) foundation models, 2025. URL <https://arxiv.org/abs/2508.06471>.
- [88] Kimi Team, Yifan Bai, Yiping Bao, Guanduo Chen, Jiahao Chen, Ningxin Chen, Ruijue Chen, Yanru Chen, Yuankun Chen, Yutian Chen, Zhuofu Chen, Jialei Cui, Hao Ding, Mengnan Dong, Angang Du, Chenzhuang Du, Dikang Du, Yulun Du, Yu Fan, Yichen Feng, Kelin Fu, Bofei Gao, Hongcheng Gao, Peizhong Gao, Tong Gao, Xinran Gu, Longyu Guan, Haiqing Guo, Jianhang Guo, Hao Hu, Xiaoru Hao, Tianhong He, Weiran He, Wenyang He, Chao Hong, Yangyang Hu, Zhenxing Hu, Weixiao Huang, Zhiqi Huang, Zihao Huang, Tao Jiang, Zhejun Jiang, Xinyi Jin, Yongsheng Kang, Guokun Lai, Cheng Li, Fang Li, Haoyang Li, Ming Li, Wentao Li, Yuanhao Li, Yiwei Li, Zhaowei Li, Zheming Li, Hongzhan Lin, Xiaohan Lin, Zongyu Lin, Chengyin Liu, Chenyu Liu, Hongzhang Liu, Jingyuan Liu, Junqi Liu, Liang Liu, Shaowei Liu, T. Y. Liu, Tianwei Liu, Weizhou Liu, Yangyang Liu, Yibo Liu, Yiping Liu, Yue Liu, Zhengying Liu, Enzhe Lu, Lijun Lu, Shengling Ma, Xinyu Ma, Yingwei Ma, Shaoguang Mao, Jie Mei, Xin Men, Yibo Miao, Siyuan Pan, Yebo Peng, Ruoyu Qin, Bowen Qu, Zeyu Shang, Lidong Shi, Shengyuan Shi, Feifan Song, Jianlin Su, Zhengyuan Su, Xinjie Sun, Flood Sung, Heyi Tang, Jiawen Tao, Qifeng Teng, Chensi Wang, Dinglu Wang, Feng Wang, Haiming Wang, Jianzhou Wang, Jiaxing Wang, Jinhong Wang, Shengjie Wang, Shuyi Wang, Yao Wang, Yejie Wang, Yiqin Wang, Yuxin Wang, Yuzhi Wang, Zhaoji Wang, Zhengtao Wang, Zhexu Wang, Chu Wei, Qianqian Wei, Wenhao Wu, Xingzhe Wu, Yuxin Wu, Chenjun Xiao, Xiaotong Xie, Weimin Xiong, Boyu Xu, Jing Xu, Jinjing Xu, L. H. Xu, Lin Xu, Suting Xu, Weixin Xu, Xinran Xu, Yangchuan Xu, Ziyao Xu, Junjie Yan, Yuzi Yan, Xiaofei Yang, Ying Yang, Zhen Yang, Zhilin Yang, Zonghan Yang, Haotian Yao, Xingcheng Yao, Wenjie Ye, Zhuorui Ye, Bohong Yin, Longhui Yu, Enming Yuan, Hongbang Yuan, Mengjie Yuan, Haobing Zhan, Dehao Zhang, Hao Zhang, Wanlu Zhang, Xiaobin Zhang, Yangkun Zhang, Yizhi Zhang, Yongting Zhang, Yu Zhang, Yutao Zhang, Yutong Zhang, Zheng Zhang, Haotian Zhao, Yikai Zhao, Huabin Zheng, Shaojie Zheng, Jianren Zhou, Xinyu Zhou, Zaida Zhou, Zhen Zhu, Wei Yu Zhuang, and Xinxing Zu. Kimi k2: Open agentic intelligence, 2025. URL <https://arxiv.org/abs/2507.20534>.
- [89] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. URL <https://arxiv.org/abs/2110.14168>.

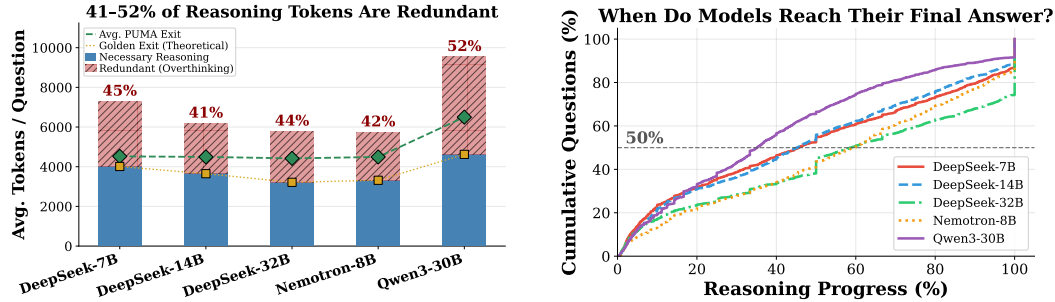
- [90] Etash Kumar Guha, Ryan Marten, Sedrick Keh, Negin Raoof, Georgios Smyrnis, Hritik Bansal, Marianna Nezhurina, Jean Mercat, Trung Vu, Zayne Rea Sprague, Ashima Suvarna, Benjamin Feuer, Leon Liangyu Chen, Zaid Khan, Eric Frankel, Sachin Grover, Caroline Choi, Niklas Muennighoff, Shiye Su, Wanjia Zhao, John Yang, Shreyas Pimpalgaonkar, Kartik sharma, Charlie Cheng-Jie Ji, Yichuan Deng, Sarah M Pratt, Vivek Ramanujan, Jon Saad-Falcon, Stutee Acharya, Jeffrey Li, Achal Dave, Alon Albalak, Kushal Arora, Blake Wulfe, Chinmay Hegde, Greg Durrett, Sewoong Oh, Mohit Bansal, Saadia Gabriel, Aditya Grover, Kai-Wei Chang, Vaishaal Shankar, Aaron Gokaslan, Mike A Merrill, Tatsunori Hashimoto, Yejin Choi, Jenia Jitsev, Reinhard Heckel, Maheswaran Sathiamoorthy, Alex Dimakis, and Ludwig Schmidt. Openthoughts: Data recipes for reasoning models. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=7xjoTuaNmN>.
- [91] Yuze Zhao, Jintao Huang, Jinghan Hu, Xingjun Wang, Yunlin Mao, Daoze Zhang, Zeyinzi Jiang, Zhikai Wu, Baole Ai, Ang Wang, Wenmeng Zhou, and Yingda Chen. Swift:a scalable lightweight infrastructure for fine-tuning, 2024. URL <https://arxiv.org/abs/2408.05517>.

A Overthinking Analysis and Failure Modes of Answer-Level Signals

A.1 Overthinking Prevalence

To quantify the extent of overthinking, we perform a counterfactual analysis on full CoT reasoning traces across five representative LRMs and five benchmarks. For each question, we identify the *golden step*: the earliest reasoning step at which the model’s intermediate trial answer matches its final answer. The tokens generated after the golden step are classified as post-answer redundancy.

Figure 5 summarizes the results. Across all five models, 41–52% of reasoning tokens are generated after the model has already committed to its final answer. The pattern is consistent across model families and scales: even the strongest model (Qwen3-30B-A3B-Thinking) spends over half its tokens on post-answer re-verification, rephrasing, and re-derivation. The cumulative distribution in Figure 5b further shows that models typically commit to their final answer well before the reasoning chain completes, with most models reaching the 50% mark between 40–60% of reasoning progress.



(a) Token breakdown before and after the model first reaches its final answer.

(b) Cumulative fraction of questions whose final answer has already been reached, as a function of reasoning progress.

Figure 5: **Overthinking analysis across five representative LRMs.** Left: a substantial fraction of reasoning tokens is generated after the model has already reached its final answer, accounting for 41–52% of tokens across models. Right: many examples reach the final answer well before the end of the reasoning chain, further illustrating the prevalence of post-answer overthinking.

A.2 Failure Rates of Answer-Level Stopping Signals

We analyze the failure rates of the two dominant classes of answer-level stopping signals by applying their criteria retroactively to full CoT traces. At each reasoning step, we induce a trial answer and compute its confidence score as the geometric mean of token-level log-probabilities, following DEER [32], and check trial-answer consistency across consecutive steps [35, 33].

We evaluate across five models and five benchmarks. For the confidence signal, we flag a step as a candidate exit whenever the confidence score exceeds $\lambda = 0.95$, following DEER [32]. For the consistency signal, we flag a step whenever the model produces $k=3$ consecutive identical trial answers, following Dynasor [33]. Table 6 reports the failure rate of each signal, defined as the fraction of first-triggered exits where the trial answer is incorrect. Averaged over all 25 (model, benchmark) combinations, the failure rate is **44%** for confidence-based exits and **64%** for consistency-based exits. Failure rates are highest on the most challenging benchmarks: on AIME24 and AIME25, consistency-based exits are incorrect in up to 83% of triggered steps, while even confidence-based exits fail up to 69% of the time.

A.3 Counterfactual Analysis: Do Signal Misfires Prevent Self-Correction?

The failure rates above show how often a signal would stop at an incorrect trial answer, but they do not tell whether early stopping itself changes the final outcome: in some cases, the model may already be on a trajectory that remains wrong even if allowed to continue. To distinguish genuinely premature exits from non-recoverable failures, we compare each failed first-triggered exit against the corresponding full, uninterrupted reasoning chain.

Table 6: Failure rates (%) of answer-level stopping signals across benchmarks and model families. A failure is defined as a first-triggered exit where the trial answer is incorrect.

	AIME24	AIME25	MATH-500	OlymBench	GPQA-D	Avg.
DeepSeek-R1-Distill-Qwen-7B						
Confidence	50.0	63.3	31.9	59.3	38.4	48.6
Consistency	76.9	70.0	36.5	67.9	67.2	63.7
DeepSeek-R1-Distill-Qwen-14B						
Confidence	53.3	62.1	28.8	52.3	35.3	46.4
Consistency	80.0	79.3	37.1	70.5	61.7	65.7
DeepSeek-R1-Distill-Qwen-32B						
Confidence	34.5	56.7	19.9	46.8	26.8	36.9
Consistency	69.0	72.4	45.5	70.1	59.6	63.3
Llama-3.1-Nemotron-Nano-8B						
Confidence	44.8	69.0	32.7	57.3	54.2	51.6
Consistency	83.3	83.3	34.9	67.9	77.3	69.3
Qwen3-30B-A3B-Thinking						
Confidence	29.6	51.9	26.3	48.4	36.5	38.5
Consistency	73.3	67.9	31.1	64.0	55.3	58.3
<i>All models avg.</i>						
Confidence	42.4	60.6	27.9	52.8	38.2	44.4
Consistency	76.5	74.6	37.0	68.1	64.2	64.1

Table 7: Counterfactual analysis of answer-level stopping signals, aggregated across all five benchmarks. **Premature%** denotes the fraction of incorrect exits where the corresponding uninterrupted Full-CoT trajectory would eventually reach the correct answer, meaning that early stopping would prevent later self-correction and induce accuracy loss.

Model	Premature%	
	Confidence	Consistency
DeepSeek-R1-Distill-Qwen-7B	41.1	47.8
DeepSeek-R1-Distill-Qwen-14B	39.8	54.6
DeepSeek-R1-Distill-Qwen-32B	28.7	52.0
Llama-3.1-Nemotron-Nano-8B	45.2	53.5
Qwen3-30B-A3B-Thinking	57.4	66.0
<i>Average</i>	42.4	54.8

We categorize a failed exit as a **premature exit** if the trial answer at the triggered step is incorrect but the full reasoning chain eventually reaches the correct final answer. In contrast, we categorize it as a **non-recoverable failure** if the full reasoning chain also ends with an incorrect answer. Premature exits are especially harmful because the stopping signal would convert an otherwise correct full-CoT trajectory into an incorrect early-exit output.

Table 7 reports the breakdown aggregated across all five benchmarks. A substantial fraction of signal failures are premature rather than non-recoverable: across all models, **42.4%** of confidence-based failures and **54.8%** of consistency-based failures occur on traces where the model would have self-corrected if allowed to continue. This shows that answer-level signals do not merely fire on trajectories that remain wrong; they often fire while the model is still in the process of correcting or revising its answer.

The counterfactual results also explain why trial-answer consistency can be particularly misleading. A model may produce the same wrong trial answer for several consecutive steps while it is still exploring the problem, creating the appearance of answer stability before the reasoning trajectory has actually converged. Stopping at this point prevents later correction. These findings reinforce the need for a reasoning-level signal: safe early exit should not only ask whether the current answer appears stable, but also whether the reasoning process has stopped making semantically novel progress.

A.4 Threshold Sensitivity of Answer-Level Stopping Signals

One possible concern is that the high failure rates above may simply reflect suboptimal threshold choices. To test this, we sweep each answer-level signal over a range of operating points and examine

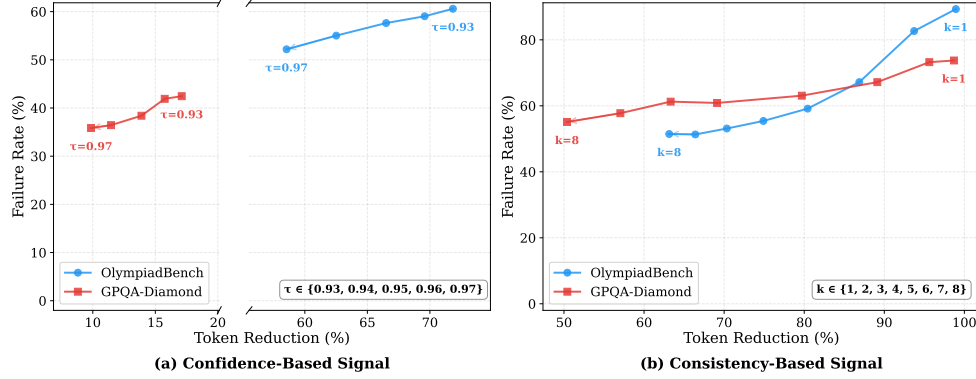


Figure 6: Threshold sensitivity of standalone answer-level stopping signals. We sweep the confidence threshold λ and the consistency window size k on OlympiadBench and GPQA Diamond using DeepSeek-R1-Distill-Qwen-7B. Each point corresponds to one operating point. Across thresholds, standalone answer-level signals exhibit a persistent tradeoff between failure rate and token reduction, without a clear setting that is both safe and efficient.

the resulting tradeoff between token reduction and failure rate. For confidence-based stopping, we vary the confidence threshold $\lambda \in \{0.93, 0.94, 0.95, 0.96, 0.97\}$. For consistency-based stopping, we vary the required number of consecutive identical trial answers $k \in \{1, 2, 3, 4, 5, 6, 7, 8\}$. We run this diagnostic analysis on DeepSeek-R1-Distill-Qwen-7B over two challenging benchmarks, OlympiadBench and GPQA Diamond.

Figure 6 shows that threshold tuning does not remove the fundamental tradeoff for standalone answer-level stopping signals. For confidence-based stopping on OlympiadBench, even conservative thresholds still yield high failure rates while stopping a large fraction of reasoning tokens. On GPQA Diamond, lower failure rates are possible only when token reduction becomes small, making early exit much less useful. Consistency-based stopping shows a similar pattern: increasing k reduces some premature triggers, but does not produce a clear operating point that is both safe and efficient. These results suggest that the limitation is not merely a poor threshold choice; standalone answer-level stopping signals are insufficient to determine whether the reasoning trajectory has actually converged.

B Implementation Details

B.1 Reasoning Step Segmentation

PUMA operates at the level of reasoning steps rather than individual tokens. We therefore segment each reasoning chain into coherent steps using a deterministic, lightweight procedure. Reasoning step segmentation is a common preprocessing step in recent work on efficient reasoning, where approaches often rely on heuristic segmentations such as blank-line splitting [54], keyword-based transition detection [68], or explicit block-marker matching [84]. Our procedure follows the blank-line approach and adds a lightweight merging stage to control step granularity. The segmentation is applied identically during Redundancy Detector training and at inference time, ensuring that the detector sees the same type of step units in both settings.

Given the raw reasoning text, we first split the chain at blank-line boundaries, which preserves the model’s natural paragraph structure. We then assign each paragraph a coarse semantic role using simple string-level cues, such as problem setup, calculation, self-correction, verification, conclusion, or general reasoning. These labels are not used as model inputs; they only guide how adjacent paragraphs are grouped into larger reasoning steps.

The final merging stage balances semantic coherence and step length. Enumerated paragraphs or major semantic transitions are treated as natural step boundaries, while short adjacent paragraphs with compatible roles are merged when doing so preserves coherence. We use a target step length range of $[L_{\min}, L_{\max}] = [200, 1000]$ characters: very short segments are often too noisy for redundancy

detection, while overly long segments may hide local repetition. Tiny trailing segments are merged into the preceding step.

The procedure is intentionally string-based and does not invoke any learned segmenter, adding negligible overhead to generation. This design keeps PUMA compatible with standard serving pipelines while providing stable step units for both detector training and online early-exit decisions.

B.2 Redundancy Detector Training

We construct a large-scale contrastive dataset to train the Redundancy Detector. The goal is to teach the detector whether a new reasoning step contributes semantically novel progress relative to prior reasoning, or instead restates, re-derives, or loops over existing content.

Source reasoning traces. We collect long-CoT reasoning traces from models that are not used as main evaluation models, including QwQ-32B [85], GPT-OSS-120B [86], GLM-4.7-Thinking [87], and Kimi-K2-Thinking [88]. These models are prompted on AMC23 and GSM8K [89], and we further include reasoning chains sampled from Open-Thoughts-114K [90]. In total, the source collection covers approximately 5,098 distinct questions. After applying the reasoning-step segmentation procedure in Appendix B.1, we obtain roughly 1.97M candidate step pairs.

Stage 1: seed annotation. We first annotate a seed set of reasoning-step pairs using GPT-5-mini. The annotation prompt asks the model to provide a short rationale and then assign a binary label, where $y = 0$ denotes a novel step and $y = 1$ denotes a redundant step. This stage yields 40,844 annotated examples across the four source models. Each example can contain multiple redundant positives and multiple novel negatives; on average, a seed example contains 3.8 positives and 26.6 negatives. After expanding multi-positive examples into single-positive contrastive rows, this stage contributes approximately 155K anchor-positive training pairs.

Stage 2: redundant-step synthesis. To increase coverage of redundant reasoning patterns, we synthesize redundant counterparts for seed-labeled novel steps. Given a question, a previous reasoning step, and a novel current step, GPT-4o-mini rewrites the current step into a redundant version that preserves the topic but removes substantive new progress. The rewrite prompt covers common redundancy modes such as restatement, hesitation, paraphrase, and transitional padding. This stage adds approximately 560K synthetic redundant pairs. The total annotation cost across both stages is around \$2,000.

Final contrastive dataset. We convert all data into the InfoNCE format used for training, where each row contains an anchor step, one redundant positive, and a set of novel negatives. After expanding multi-positive annotations, adding synthetic redundant pairs, deduplicating examples, and removing rows without valid negatives, the final dataset contains 701,641 contrastive training rows.

Training setup. We implement detector training using the MS-Swift framework [91]. Instead of full-parameter fine-tuning, we adapt Qwen3-Embedding-0.6B with LoRA to reduce overfitting risk. We use LoRA rank $r = 32$, $\alpha = 64$, dropout 0.1, and apply LoRA to the `q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, and `down_proj` modules. The detector is trained with an InfoNCE contrastive objective, batch size 16 with gradient accumulation 4 (effective batch size 64), learning rate 1×10^{-4} , and 5 epochs.

Detector quality and threshold selection. We evaluate the trained detector on a held-out set of anchor-positive-negative triples, where the positive is a redundant step and the negative is a novel step relative to the anchor. The detector achieves 91.26% pairwise ranking accuracy, where a prediction is counted as correct if

$$\text{sim}(\text{anchor}, \text{redundant}) > \text{sim}(\text{anchor}, \text{novel}). \quad (4)$$

The average cosine similarity is 0.333 for redundant pairs and 0.184 for novel pairs, yielding an average margin of 0.148. For the default PUMA setting, we set $\tau_{\text{sim}} = 0.35$, selected on a held-out calibration set as a conservative operating point for candidate-exit detection. At this threshold, the detector achieves 91.54% absolute classification accuracy and 93.58% true-negative rate. The high true-negative rate makes the detector conservative: it avoids over-triggering on genuinely novel reasoning steps, while missed redundant steps mainly reduce potential token savings rather than directly harming correctness. Moreover, all detector-flagged candidate exits are still filtered by Answer Verification before PUMA stops.

LLM Annotation Prompts for Redundancy Detector Training We use two LLM-based prompts to construct supervision for the Redundancy Detector: a seed novelty annotation prompt (Table 8) and a redundant-step synthesis prompt (Table 9). For clarity, we present both prompts under the label convention used in our final training data, where $y = 0$ denotes a novel reasoning step and $y = 1$ denotes a redundant reasoning step.

Role:

You are an AI assistant for assessing the quality of logical reasoning. The user will provide a question, a previous reasoning step, and a current reasoning step. Your job is to judge whether the current step provides additional information compared to the previous step. Do not solve the problem yourself; only evaluate the novelty of the current step.

Label Convention:

Output 0 if the current step introduces new logical or semantic information.

Output 1 if the current step is redundant, i.e., it mainly restates, rephrases, verifies, or loops over information already present in the previous step.

Output Format:

First provide a short explanation enclosed in `<explanation>...</explanation>`. Then output the final label enclosed in `<result>...</result>`. The result must be exactly one character, either 0 or 1. Output nothing after the closing `</result>` tag.

Few-shot Examples:

[Four worked demonstrations illustrating novel vs. redundant reasoning-step judgments.]

Input:

Problem: `<input question>`

Previous Step: `<previous reasoning step>`

Current Step: `<current reasoning step>`

Expected Output:

`<explanation>`

`<brief rationale explaining whether the current step is novel or redundant>`

`</explanation>`

`<result>0 or 1</result>`

Table 8: Prompt template for seed novelty annotation. The displayed label convention matches the remapped labels used in the final Redundancy Detector training data.

B.3 Choice of Redundancy Signal

Semantic Entropy commonly operationalizes semantic equivalence using Natural Language Inference (NLI), by asking whether two textual outputs mutually entail each other. Motivated by this connection, we compare PUMA’s embedding-based Redundancy Detector with two NLI-style alternatives. The first variant, **ICL-NLI**, uses Qwen3-0.6B with 4-shot in-context prompting: given two reasoning steps and four labelled examples, the model emits a binary “Yes/No” judgment for whether the current step is redundant with respect to the previous one. The second variant, **FT-NLI**, uses the same Qwen3-0.6B backbone fine-tuned on 10K NLI-style redundancy examples. For both variants, the binary judgment replaces PUMA’s default redundancy score; all other PUMA components are kept unchanged.

Table 10 reports results on DS-7B and Nemotron-8B across five benchmarks. The embedding-based detector achieves the best accuracy–efficiency trade-off: it is the only signal that preserves (and even slightly improves) accuracy over Full CoT (+1.3 on average) while still achieving meaningful token reduction (27.9%). ICL-NLI is too conservative—it rarely fires, leaving most reasoning uncompressed (8.2% TR). FT-NLI shows the opposite failure mode: it achieves 39.9% TR but drops accuracy by 3.8 points, indicating that NLI-style judgments do not align well with reasoning-level convergence even after task-specific fine-tuning.

Role:

You are an expert at analyzing mathematical reasoning steps. Your task is to generate a redundant version of a given reasoning step.

Definition:

A redundant step is one that does not advance the solution compared to the previous step. It may restate or summarize what was already said, express hesitation or self-checking without new insight, rephrase the same mathematical relationship in different words, or add transitional phrases without actual progress.

Instruction:

Given a mathematical problem, a previous reasoning step, and a current step that provides new information, generate a redundant version of the current step. The redundant version should:

- not provide new information beyond what is already in the previous step;
- sound natural as a reasoning step;
- be similar in length to the current step.

Few-shot Examples:

[Examples demonstrate common redundancy modes, including restatement, hesitation, paraphrase, and transitional padding.]

Input:

Problem: <input question>

Previous Step: <previous reasoning step>

Current Step (Novel): <current reasoning step that introduces new information>

Output:

Redundant Version: <rewritten reasoning step that sounds natural but adds no substantive new progress>

Table 9: Prompt template for redundant-step synthesis. GPT-4o-mini rewrites seed-labeled novel steps into redundant counterparts to augment positive examples for contrastive training.

Table 10: Redundancy signal and detector comparison. Fine-tuned Embedding is PUMA’s default detector: a LoRA-adapted Qwen3-Embedding-0.6B trained with an InfoNCE contrastive objective. Base Embedding uses the off-the-shelf Qwen3-Embedding-0.6B without contrastive fine-tuning, with two representative similarity thresholds. ICL-NLI and FT-NLI replace the detector with NLI-style redundancy judgments while keeping all other PUMA components unchanged. Results are averaged over five benchmarks.

Signal	DS-7B		Nemotron-8B		Average	
	$\Delta\text{Acc}\uparrow$	TR $\uparrow$	$\Delta\text{Acc}\uparrow$	TR $\uparrow$	$\Delta\text{Acc}\uparrow$	TR $\uparrow$
Fine-tuned Emb. (PUMA)	+2.2	35.6	+0.4	20.1	+1.3	27.9
Base Emb. ($\tau = 0.42$)	-2.9	46.1	-3.3	37.3	-3.1	41.7
Base Emb. ($\tau = 0.50$)	-4.1	45.5	-3.3	37.5	-3.7	41.5
ICL-NLI	-2.1	10.9	-0.1	5.5	-1.1	8.2
FT-NLI	-4.1	44.3	-3.5	35.4	-3.8	39.9

Beyond accuracy and token reduction, the embedding detector is substantially cheaper at inference time, averaging about 21 ms per question compared with ~ 93 ms for both NLI variants. These results support using the trained embedding model as PUMA’s default redundancy signal.

We additionally compare against the base (unfine-tuned) Qwen3-Embedding-0.6B to isolate the contribution of contrastive training. Base-Emb achieves higher token reduction but at a substantial accuracy cost, and varying its similarity threshold does not recover the gap, suggesting that task-specific fine-tuning is essential for reliable redundancy detection.

B.4 Redundancy Detector Lookback Window

The Redundancy Detector compares the current reasoning step against the previous k reasoning steps and flags a candidate exit when the maximum similarity exceeds τ_{sim} . PUMA uses $k = 1$ by default, comparing each step only with its immediate predecessor. Table 11 evaluates $k \in \{1, 2, 4, 8, \text{all}\}$ on DS-7B and Nemotron-8B, where $k = \text{all}$ compares the current step against all preceding steps.

Across both models, $k = 1$ provides the best accuracy-preserving behavior. Increasing k yields more aggressive exits and higher token reduction, but it also worsens ΔAcc on average, from $+1.3$ at $k = 1$ to -2.4 at $k = \text{all}$. This pattern suggests that larger windows introduce additional false-positive triggers: a current step may resemble a much earlier step while still contributing useful progress relative to the immediately preceding context. As a result, looking too far back can incorrectly trigger exits during productive reasoning. We therefore use $k = 1$ as the default because it provides the most favorable accuracy–compression tradeoff in this sweep.

Table 11: Redundancy Detector lookback window: k is the number of preceding reasoning steps the detector compares the current step against (PUMA default: $k=1$); $k=\text{all}$ compares against every prior step. Each row reports averages over five benchmarks.

k	DS-7B		Nemotron-8B		Average	
	$\Delta\text{Acc}\uparrow$	$\text{TR}\uparrow\uparrow$	$\Delta\text{Acc}\uparrow$	$\text{TR}\uparrow\uparrow$	$\Delta\text{Acc}\uparrow$	$\text{TR}\uparrow\uparrow$
1 (default)	+2.2	35.6	+0.4	20.1	+1.3	27.9
2	+0.0	37.7	-0.8	25.5	-0.4	31.6
4	-1.7	40.0	-3.3	28.3	-2.5	34.2
8	-0.5	41.6	-1.5	31.2	-1.0	36.4
all	-2.0	46.4	-3.8	37.5	-2.9	41.9

B.5 A Semantic-Entropy Perspective on Reasoning Convergence

We provide a semantic-entropy perspective to explain why reasoning-step redundancy is a useful signal for convergence. Semantic Entropy [36, 37] measures uncertainty in LLM outputs through semantic diversity rather than surface-form variation. We adapt this intuition from multiple sampled outputs to successive steps within a single reasoning trajectory: if recent steps continue to introduce semantically distinct content, the model is still exploring; if they collapse into repeated verification, restatement, or re-derivation, the trajectory has likely entered a locally converged phase. To formalize this view, consider a recent reasoning window $W_t^{(w)} = (r_{t-w+1}, \dots, r_t)$ and let $c(r)$ denote the semantic cluster associated with reasoning step r . We define the *Reasoning Semantic Entropy* of this window as

$$\text{RSE}_w(t) = - \sum_c p_t(c) \log p_t(c), \quad (5)$$

where $p_t(c)$ is the fraction of steps in $W_t^{(w)}$ assigned to cluster c . High $\text{RSE}_w(t)$ indicates semantically diverse recent reasoning and continued exploration, whereas low $\text{RSE}_w(t)$ indicates that recent steps have collapsed into a small set of redundant semantic patterns. Thus, low local RSE corresponds to local reasoning convergence.

PUMA does not explicitly compute $\text{RSE}_w(t)$ online. Doing so would require clustering recent reasoning steps, for example using entailment-based semantic equivalence as in Semantic Entropy, which would add substantial overhead for deployment-time early exit. Instead, PUMA uses the Redundancy Detector as a lightweight online proxy for local semantic collapse: high similarity between recent steps suggests that the reasoning trajectory is becoming less semantically diverse and less likely to be making novel progress. This provides an interpretive lens for PUMA: the Redundancy Detector approximates the local semantic collapse that low RSE would capture, without explicitly computing semantic entropy.

B.6 Trial Answer Induction

At each candidate stopping point, PUMA appends a *confident ending* suffix to the current reasoning prefix and prompts the model to commit to a trial answer. The suffix is task-specific:

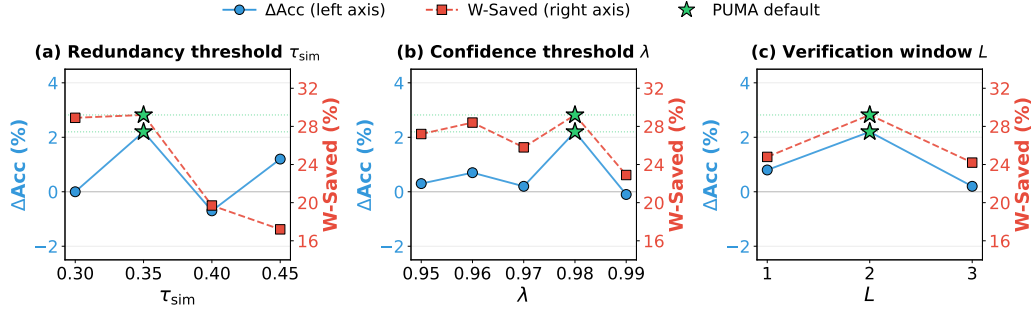


Figure 7: Sensitivity to PUMA’s three core stopping hyperparameters on DS-7B, averaged over five benchmarks. Star markers and dashed horizontal lines indicate the default configuration.

- **Math** (AIME, MATH-500, OlympiadBench): `\n**Final Answer**\n\nThe final answer is \boxed{`
- **Multiple choice** (GPQA-Diamond): `\n**Final Answer**\n\nThe answer choice is \boxed{`
- **Code** (LiveCodeBench): `</think>\n\n### Solution Code\n\n“python\n`

For math and multiple-choice tasks, the suffix intentionally does not close the `</think>` tag, keeping the model in thinking mode so that it commits to an answer without switching to an explanation mode that might introduce new reasoning. For code, we close the tag explicitly because code answers require the model to generate a complete Python program in solution mode.

Confidence computation. For math tasks, we extract the token span inside `\boxed{...}` using brace matching and compute confidence as the geometric mean of token-level log-probabilities over the answer tokens (excluding braces), following DEER [32]. For GPQA, the single answer token (A/B/C/D) competes against the full vocabulary, yielding artificially low absolute log-probabilities. We therefore apply a temperature-scaled softmax restricted to the four answer choices to obtain a calibrated confidence score. For code, we use the log-probabilities of all generated tokens, since code answers lack a `\boxed{}` structure.

Answer consistency. Trial answers across the verification window are compared via exact string match for math and multiple-choice tasks. For code, we use fuzzy matching via Python’s `difflib.SequenceMatcher` (sequence similarity ≥ 0.8) to accommodate minor whitespace and formatting differences.

Generation is capped at ~ 30 tokens for math, which is typically sufficient for a complete boxed answer. For code, we cap at ~ 50 tokens, which rarely covers a full program but suffices to compute a meaningful confidence estimate from the opening tokens of the solution. Generating complete code trial answers could require hundreds of tokens per probe, making the overhead prohibitive for an online early-exit system.

B.7 Extended Hyperparameter Analysis

Figure 7 sweeps PUMA’s three core stopping hyperparameters on DS-7B across five benchmarks. A notable finding is that PUMA is robust across a wide range of settings: out of 9 alternative configurations tested, only two produce a negative ΔAcc (-0.7 at $\tau_{sim}=0.40$ and -0.1 at $\lambda=0.99$), and both are within 1pp of Full CoT. This robustness reflects PUMA’s layered safety design: even when one hyperparameter is set suboptimally, the remaining components (Redundancy Detector, Answer Verification, Loop Breaker) continue to prevent most unsafe exits.

The redundancy threshold τ_{sim} is the most safety-critical parameter. Lowering it to 0.30 maintains token savings but eliminates the accuracy margin (ΔAcc drops from +2.2 to 0.0), while raising it to 0.40 over-suppresses the redundancy signal, nearly halving token savings without recovering accuracy. The confidence threshold λ is stable across $[0.95, 0.98]$, with only minor accuracy–efficiency variation. The sharp drop at 0.99 (-6.3pp TR) indicates that requiring near-perfect trial-answer confidence causes the verifier to veto most candidate exits. The verification window L controls how much evidence PUMA requires before stopping. $L = 1$ exits after the first verified candidate and is

Table 12: Dataset statistics for all evaluation benchmarks.

Benchmark	# Examples	Domain
<i>Main reasoning benchmarks</i>		
MATH-500 [41]	500	Competition mathematics
AIME24 [42]	30	Contest mathematics
AIME25 [43]	30	Contest mathematics
OlympiadBench [44]	675	Olympiad-level math and physics
GPQA-Diamond [45]	198	Graduate-level science QA
<i>Generalization benchmarks</i>		
LiveCodeBench [79]	880	Code generation
MathVista [80]	200	Vision-language mathematical reasoning
MathVision [81]	200	Vision-language mathematical reasoning

therefore more susceptible to single-step false positives. $L = 3$ is overly conservative: the additional verified candidate often does not appear before generation continues, reducing token reduction without improving accuracy. The default $L = 2$ provides the best balance.

The main stopping hyperparameters (τ_{sim} , λ , L) are shared across all models. The Loop Breaker consecutive-redundancy threshold m is configured per model based on validation performance on a held-out AMC23 set, as different models exhibit different redundancy patterns in late-stage reasoning. DS-R1-Distill-Qwen-7B and Nemotron-Nano-8B use $m=1$; DS-R1-Distill-Qwen-14B uses $m=3$; DS-R1-Distill-Qwen-32B uses $m=4$. For Qwen3-30B-A3B-Thinking, the Loop Breaker is not activated, as verified exits alone provide sufficient token savings.

C Experimental Details and Full Results

C.1 Benchmark and Dataset Statistics

Table 12 summarizes the benchmarks used in our experiments. The main evaluation covers five challenging reasoning benchmarks spanning competition mathematics, olympiad-level STEM, and graduate-level science. For OlympiadBench, we use the standard text-only English open-ended math subset (675 problems). Generalization experiments additionally evaluate on code generation (LiveCodeBench) and vision-language reasoning (MathVista and MathVision). For MathVista, we evaluate on the first 200 problems from the testmini split (1,000 problems total). For MathVision, we evaluate on the first 200 problems from the full test set (3,040 problems total).

C.2 Existing Assets and Licenses

Table 13 summarizes the main existing assets used in this work and their licenses. All assets are publicly available and cited with their original references. We use these assets for research evaluation and method development, and do not redistribute third-party model weights or benchmark datasets as part of this work.

C.3 Full Main Results Across Five Models

Table 1 in the main text reports results for three models. Table 14 extends this to all five models, including DS-R1-Distill-Qwen-14B, and reports per-benchmark token counts alongside accuracy and token reduction. Figure 8 further reports the average number of reasoning steps before and after PUMA’s early-exit intervention. PUMA saves 17.6–28.5% of reasoning steps across models, with deeper compression on longer chains (DS-7B: 28.5%, Qwen3-30B-T: 27.9%) and more conservative savings on shorter chains (DS-32B: 17.6%). No-Think is a prompt-only baseline that asks the model to skip explicit reasoning, but it does not impose a decoding-time stopping rule or a hard length budget. Its effectiveness therefore depends strongly on model-specific instruction following and reasoning-format behavior. On Llama-3.1-Nemotron-Nano-8B, the No-Think prompt often fails to suppress long-form generation, leading to outputs that are longer than Full CoT and substantially less accurate. This explains the negative token reduction observed for No-Think on Nemotron and

Table 13: Licenses for main existing assets used in this work.

Asset	Type	License
DeepSeek-R1-Distill	Model	MIT
Llama-3.1-Nemotron-Nano-8B	Model	NVIDIA Open Model License
Qwen3-30B-A3B-Thinking	Model	Apache 2.0
Qwen3-Embedding-0.6B	Model	Apache 2.0
MATH-500	Dataset	MIT
AIME24/25	Dataset	Apache 2.0
OlympiadBench	Dataset	Apache 2.0
GPQA-Diamond	Dataset	CC-BY 4.0
LiveCodeBench	Dataset	MIT
MathVista	Dataset	CC-BY-SA 4.0
MathVision	Dataset	MIT
vLLM	Software	Apache 2.0
MS-Swift	Software	Apache 2.0

highlights a limitation of prompt-only compression: asking a reasoning model to skip thinking is not a reliable substitute for monitoring and stopping the reasoning process online.

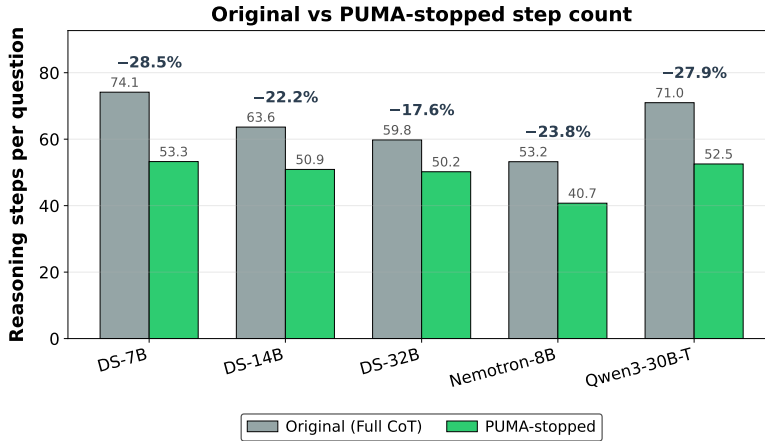


Figure 8: Reasoning-step count per question: original Full-CoT (gray) vs PUMA-stopped (green), averaged over five benchmarks. Annotated percentages report $(\text{orig} - \text{stopped})/\text{orig}$.

C.4 Full Component Ablation

Table 4 in the main text reports a compact ablation on DS-7B. Table 15 extends the analysis to both DS-7B and Qwen3-30B-A3B-Thinking, reports trial-answer probe statistics, and includes two additional variants: *AV only* and *w/o AC and CG*. *Probes/q* is the average number of trial-answer probes per question, and $\text{Probe} \times$ is normalized by full PUMA on the same model.

The full results provide a more detailed view of how each component affects accuracy, token reduction, and probing overhead. First, disabling the RD gate causes Answer Verification to be invoked at every eligible generated reasoning step. This increases token reduction, but also lowers accuracy and increases probe overhead substantially: $3.3\times$ more probes on DS-7B and $4.3\times$ more probes on Qwen3-30B-T. This shows that answer-level verification without redundancy-based candidate filtering is both less reliable and more expensive. Second, the *AV only* variant disables both the RD gate and Loop Breaker, leaving Answer Verification as the only stopping mechanism. Although it can still reduce tokens, it requires $4.0\text{--}4.3\times$ more probes than full PUMA and does not match the full model’s accuracy–efficiency balance, confirming the value of reasoning-level stopping signals. Third, removing the Loop Breaker has model-dependent effects: it sharply reduces token savings on DS-7B, while having smaller effect on Qwen3-30B-T, suggesting that different LRMs rely on

Table 14: Full results across all five reasoning models and five benchmarks. Acc (% , $\uparrow$): accuracy; Tok ($\downarrow$): average output tokens per question; TR (% , $\uparrow$): token reduction percentage relative to Full CoT.

Method	MATH-500			AIME24			AIME25			GPQA-D			OlymBench			Overall	
	Acc $\uparrow$	Tok $\downarrow$	TR $\uparrow$	Acc $\uparrow$	Tok $\downarrow$	TR $\uparrow$	Acc $\uparrow$	Tok $\downarrow$	TR $\uparrow$	Acc $\uparrow$	Tok $\downarrow$	TR $\uparrow$	Acc $\uparrow$	Tok $\downarrow$	TR $\uparrow$	Acc $\uparrow$	TR $\uparrow$
DeepSeek-R1-Distill-Qwen-7B																	
Full CoT	90.0	4145	0.0	50.0	14280	0.0	43.3	15040	0.0	49.0	7153	0.0	57.6	8988	0.0	58.0	0.0
No-Think	79.0	830	80.0	20.0	4615	67.7	23.3	6326	57.9	28.3	904	87.3	42.8	1882	79.1	38.7	79.9
CCoT	85.0	1622	60.9	36.7	5775	59.6	33.3	5989	60.2	48.0	3178	55.6	49.2	3804	57.7	50.4	58.6
CoD	80.4	1662	59.9	40.0	6151	56.9	33.3	6214	58.7	50.5	3518	50.8	43.0	3973	55.8	49.4	56.6
Plan&Budget	84.6	2133	48.5	43.3	6722	52.9	23.3	6938	53.9	38.4	4565	36.2	49.3	4470	50.3	47.8	47.9
Ans. Conv.	61.8	974	76.5	26.7	2390	83.3	20.0	2583	82.8	34.3	544	92.4	32.6	1550	82.8	35.1	81.9
Dynasor	84.2	2878	30.6	23.3	11175	21.7	33.3	13677	9.1	39.4	2388	66.6	49.0	5923	34.1	45.9	36.6
DEER	90.6	2503	39.6	40.0	12261	14.1	50.0	13058	13.2	51.5	8398	-17.4	57.5	7171	20.2	57.9	21.5
PUMA (ours)	89.6	2546	38.6	60.0	9957	30.3	46.7	10558	29.8	49.0	6866	4.0	55.6	5107	43.2	60.2	35.6
DeepSeek-R1-Distill-Qwen-14B																	
Full CoT	91.0	3526	0.0	63.3	10209	0.0	50.0	12663	0.0	52.5	6132	0.0	61.5	7678	0.0	63.7	0.0
No-Think	80.8	1010	71.3	46.7	8689	14.9	40.0	9964	21.3	35.4	966	84.2	45.5	3348	56.4	49.7	63.8
CCoT	89.6	1771	49.8	50.0	5873	42.5	33.3	6376	49.6	57.6	3352	45.3	52.3	3970	48.3	56.6	48.3
CoD	88.8	2211	37.3	53.3	6189	39.4	30.0	6581	48.0	52.5	4009	34.6	54.8	4441	42.2	55.9	39.5
Plan&Budget	90.2	2703	23.3	43.3	6755	33.8	36.7	7059	44.3	55.0	4803	21.7	53.8	4898	36.2	55.8	29.8
Ans. Conv.	59.0	858	75.7	26.7	1947	80.9	13.3	2805	77.8	42.4	332	94.6	27.9	1117	85.5	33.9	83.1
Dynasor	89.2	3606	-2.3	56.7	10561	-3.5	53.3	13398	-5.8	58.6	4253	30.6	60.1	6805	11.4	63.6	8.6
DEER	91.2	2608	26.0	70.0	9936	2.7	50.0	12397	2.1	60.6	6918	-12.8	62.1	6950	9.5	66.8	11.9
PUMA (ours)	91.2	2776	21.3	66.7	8549	16.3	50.0	11788	6.9	55.6	5270	14.1	59.6	5229	31.9	64.6	24.9
DeepSeek-R1-Distill-Qwen-32B																	
Full CoT	89.2	2957	0.0	76.7	10011	0.0	63.3	12606	0.0	62.6	6186	0.0	59.3	7241	0.0	70.2	0.0
No-Think	84.2	1264	57.2	56.7	5781	42.2	36.7	7435	41.0	47.0	1196	80.7	49.3	4060	43.9	54.8	53.5
CCoT	87.6	1521	48.6	60.0	5600	44.1	26.7	6162	51.1	60.6	2968	52.0	53.5	3356	53.6	57.7	51.4
CoD	86.4	1558	47.3	56.7	5390	46.2	36.7	6103	51.6	64.7	3277	47.0	53.6	3724	48.6	59.6	47.9
Plan&Budget	90.8	2462	16.7	56.7	6257	37.5	40.0	6492	48.5	59.6	4335	29.9	54.1	4522	37.5	60.2	29.4
Ans. Conv.	65.6	631	78.7	16.7	1244	87.6	6.7	1638	87.0	37.9	310	95.0	30.2	889	87.7	31.4	85.6
Dynasor	92.0	3651	-23.5	53.3	10120	-1.1	50.0	12822	-1.7	58.1	4121	33.4	64.6	7050	2.6	63.6	-2.4
DEER	94.2	2424	18.0	66.7	10439	-4.3	46.7	11753	6.8	67.2	6222	-0.6	63.1	6804	6.0	67.6	9.1
PUMA (ours)	88.4	2296	22.3	73.3	9046	9.6	60.0	11144	11.6	66.7	5394	12.8	59.9	5344	26.2	69.7	22.3
Llama-3.1-Nemotron-Nano-8B																	
Full CoT	93.6	3109	0.0	66.7	10463	0.0	50.0	10898	0.0	48.0	7001	0.0	63.1	6857	0.0	64.3	0.0
No-Think	62.8	7012	-125.5	26.7	13842	-32.3	16.7	15628	-43.4	25.8	7154	-2.2	31.9	11935	-74.0	32.8	-80.5
CCoT	86.6	2035	34.5	36.7	6701	36.0	20.0	6966	36.1	43.9	4824	31.1	56.9	4485	34.6	48.8	34.1
CoD	79.0	2173	30.1	50.0	6189	40.9	33.3	6610	39.4	45.0	4766	31.9	51.3	4665	32.0	51.7	31.7
Plan&Budget	89.0	2852	8.3	43.3	7022	32.9	33.3	7460	31.6	40.9	5133	26.7	53.9	5299	22.7	52.1	18.6
Ans. Conv.	55.8	735	76.3	6.7	849	91.9	3.3	1050	90.4	20.2	331	95.3	23.6	1024	85.1	21.9	83.7
Dynasor	91.4	3388	-9.0	50.0	10572	-1.0	43.3	12467	-14.4	45.0	6894	1.5	61.9	7059	-3.0	58.3	-4.7
DEER	91.8	2403	22.7	53.3	12814	-22.5	50.0	14805	-35.9	49.5	28931	-313.2	61.2	6004	12.4	61.2	-30.7
PUMA (ours)	92.6	2621	15.7	70.0	9180	12.3	50.0	8928	18.1	48.5	6349	9.3	62.2	5007	27.0	64.7	20.1
Qwen3-30B-A3B-Thinking																	
Full CoT	94.4	5206	0.0	83.3	16354	0.0	83.3	18473	0.0	72.7	7372	0.0	75.0	12668	0.0	81.7	0.0
No-Think	91.8	2516	51.7	60.0	7107	56.5	50.0	9246	50.0	73.7	8221	-11.5	68.4	5503	56.6	68.8	45.3
CCoT	89.2	2361	54.6	36.7	7217	55.9	20.0	7341	60.3	72.7	4021	45.5	53.0	5220	58.8	54.3	55.5
CoD	82.8	3652	29.9	23.3	7770	52.5	10.0	7788	57.8	65.2	5247	28.8	45.3	6297	50.3	45.3	40.4
Plan&Budget	91.8	2532	51.4	43.3	7100	56.6	36.7	7306	60.5	64.7	4635	37.1	61.3	5039	60.2	59.6	53.9
Ans. Conv.	57.0	635	87.8	0.0	921	94.4	0.0	1738	90.6	35.9	333	95.5	23.1	1050	91.7	23.2	90.9
Dynasor	94.8	5432	-4.3	86.7	14372	12.1	76.7	17959	2.8	67.7	5001	32.2	72.4	12748	-0.6	79.7	3.0
DEER	94.8	3416	34.9	83.3	14682	10.5	80.0	17181	7.3	74.8	8062	-8.2	73.5	9749	23.4	81.3	22.4
PUMA (ours)	94.2	3707	28.8	90.0	12914	21.0	80.0	15767	14.7	75.8	6058	17.8	72.3	8652	31.7	82.5	28.2

the fallback to different degrees. Finally, disabling the answer-verification gates shows their role in preventing premature exits. Removing either Answer Consistency or the Confidence Gate degrades accuracy, and disabling both gates causes the largest accuracy collapse despite high token reduction. Overall, the full ablation shows that PUMA’s components are complementary: the RD gate controls when verification is invoked, Answer Verification filters unreliable candidates, and the Loop Breaker provides additional savings when reasoning enters sustained redundancy.

C.5 Detailed Latency Analysis

Figure 3 in the main text reports average speedup across benchmarks. Table 16 provides per-benchmark wall-clock latency for three models. PUMA achieves consistent speedup across all settings (1.15–1.40 $\times$), while DEER is uniformly slower than Full CoT (0.52–0.71 $\times$) and Dynasor ranges from 1.11 $\times$ to 0.17 $\times$ depending on model size. These results show that token reduction alone does not guarantee wall-clock gains. Both DEER and Dynasor can reduce generated tokens yet still run slower than Full CoT, because frequent trial-answer probing introduces additional forward-pass

Table 15: Full component ablation and probe overhead on DS-R1-Distill-Qwen-7B and Qwen3-30B-A3B-Thinking, averaged over AIME24, AIME25, GPQA-Diamond, MATH-500, and Olympiad-Bench. *Probes/q* is the average number of trial-answer probes per question, and *Probe×* is normalized by full PUMA on the same model. “w/o RD Gate” disables redundancy-based candidate filtering by setting the redundancy threshold to zero, causing Answer Verification to be invoked at every step.

Configuration	DS-7B				Qwen3-30B-T			
	Acc↑	TR↑	Probes/q↓	Probe×	Acc↑	TR↑	Probes/q↓	Probe×
PUMA (full)	60.2	35.6	13.75	1.0×	82.5	28.2	10.46	1.0×
w/o RD Gate	56.1 (-4.1)	46.0	45.69	3.3×	78.6 (-3.9)	42.9	45.11	4.3×
w/o Loop Breaker	59.2 (-1.0)	22.6	17.48	1.3×	80.5 (-2.0)	27.4	11.88	1.1×
AV only (w/o RD and LB)	58.3 (-1.9)	37.3	55.27	4.0×	77.6 (-4.9)	42.8	45.13	4.3×
w/o Answer Consistency	57.3 (-2.9)	37.8	15.96	1.2×	79.9 (-2.6)	34.7	9.04	0.9×
w/o Confidence Gate	53.7 (-6.5)	55.3	8.64	0.6×	72.6 (-9.9)	47.9	5.11	0.5×
w/o AC and CG	44.0 (-16.2)	66.1	4.55	0.3×	61.9 (-20.6)	55.8	2.95	0.3×

Table 16: Per-benchmark wall-clock latency across three models. For each benchmark we report accuracy (Acc, %, ↑), token reduction (TR, %, ↑), and average wall-clock seconds per question (*s/q*, ↓). Overall Speedup is computed from total wall-clock time relative to Full CoT. Speedup below 1× (in red) indicates slower wall-clock performance than Full CoT. For the *s/q* columns, the **best** (lowest) and **second-best** values within each model are highlighted.

Method	MATH-500			AIME24			AIME25			OlympiadBench			Overall			
	Acc↑	TR↑	<i>s/q</i> ↓	Acc↑	TR↑	<i>s/q</i> ↓	Acc↑	TR↑	<i>s/q</i> ↓	Acc↑	TR↑	<i>s/q</i> ↓	Acc↑	TR↑	<i>s/q</i> ↓	Speedup↑
DeepSeek-R1-Distill-Qwen-7B																
Full CoT	90.0	0.0	0.98	50.0	0.0	10.18	43.3	0.0	10.56	57.6	0.0	2.57	60.2	0.0	2.30	1.00×
DEER	90.6	39.6	1.63	40.0	14.1	12.64	50.0	13.2	12.80	57.5	20.2	3.78	59.5	27.7	3.35	0.69×
Dynasor	84.2	30.6	1.03	23.3	21.7	11.86	33.3	9.1	15.26	49.0	34.1	1.84	47.5	31.8	2.08	1.11×
PUMA (ours)	89.6	38.6	0.87	60.0	30.3	9.41	46.7	29.8	6.88	55.6	43.2	1.64	63.0	40.7	1.64	1.40×
DeepSeek-R1-Distill-Qwen-14B																
Full CoT	91.0	0.0	2.09	63.3	0.0	20.20	50.0	0.0	20.95	61.5	0.0	7.32	66.5	0.0	5.84	1.00×
DEER	91.2	26.0	3.68	70.0	2.7	22.21	50.0	2.1	23.88	62.1	9.5	10.36	68.3	15.8	8.27	0.71×
Dynasor	89.2	-2.3	8.59	56.7	-3.5	12.54	53.3	-5.8	25.09	60.1	11.4	42.96	64.8	5.1	27.87	0.21×
PUMA (ours)	91.2	21.3	1.84	66.7	16.3	15.62	50.0	6.9	20.92	59.6	31.9	5.39	66.9	26.6	4.58	1.28×
Llama-3.1-Nemotron-Nano-8B																
Full CoT	93.6	0.0	1.07	66.7	0.0	7.68	50.0	0.0	9.82	63.1	0.0	2.83	68.3	0.0	2.41	1.00×
DEER	91.8	22.7	1.83	53.3	-22.5	23.10	50.0	-35.9	26.61	61.2	12.4	4.99	64.1	14.5	4.67	0.52×
Dynasor	91.4	-9.0	0.99	50.0	-1.0	9.17	43.3	-14.4	11.37	61.9	-3.0	24.32	61.7	-5.7	14.19	0.17×
PUMA (ours)	92.6	15.7	0.85	70.0	12.3	8.71	50.0	18.1	11.31	62.2	27.0	2.30	68.7	21.9	2.09	1.15×

overhead. When this overhead outweighs the time saved from shorter reasoning traces, positive token reduction does not translate into end-to-end speedup.

D Budget Tuning Does Not Rescue Prompt-Based Baselines

A natural question is whether the accuracy gap between PUMA and prompt-based baselines can be closed by relaxing their word-budget instructions. We therefore sweep the word budgets specified in the prompts for CCoT and CoD on DS-7B LiveCodeBench. We note that LRMs do not always strictly follow these requested budgets; thus, the reported token reduction reflects the actual generated outputs rather than the nominal prompt budget. Table 17 shows that budget tuning does not close the gap: CCoT remains in the 42.7–45.8% accuracy range across requested budgets from 30 to 300 words, and CoD remains in the 43.2–44.9% range across requested budgets from 5 to 30 words per step, both below PUMA’s 50.3%.

E Reasoning Quality Evaluation Details

We evaluate the quality of retained reasoning chains using an LLM-as-Judge protocol. The goal is to measure whether an early-exit method preserves a readable and sufficient reasoning chain, beyond merely preserving final-answer accuracy.

Table 17: Budget-sweep analysis of prompt-based baselines on DS-7B LiveCodeBench. Relaxing CCoT’s global word budget or CoD’s per-step word budget does not close the accuracy gap to PUMA. Δ is accuracy change relative to Full CoT. “def.” denotes the default budget used in the main experiments.

Metric	Reference		CCoT: global word budget					CoD: per-step word budget		
	Full CoT	PUMA	30	45 (def.)	75	150	300	5 (def.)	10	30
Acc $\uparrow$	51.82	50.34	45.57	45.80	42.73	44.20	44.32	43.18	43.52	44.89
$\Delta\uparrow$	—	-1.48	-6.25	-6.02	-9.09	-7.62	-7.50	-8.64	-8.30	-6.93
TR $\uparrow$	0.0	19.0	42.9	42.7	41.8	41.4	40.0	40.5	40.6	41.3

Judge model and protocol. We use GPT-5.4-thinking as the judge via the OpenAI Batch API. Each instance contains the original question and the retained reasoning chain produced by the evaluated method. The judge is instructed to first provide a brief rationale and then assign scores, following standard LLM-as-Judge practice [73]. The judge prompt template is summarized in Table 18.

Scope. We evaluate three (model, benchmark) combinations: DS-R1-Distill-Qwen-7B on GPQA-Diamond (198 questions), Nemotron-Nano-8B on GPQA-Diamond (198 questions), and Qwen3-30B-A3B-Thinking on MATH-500 (500 questions). All questions are evaluated regardless of answer correctness, avoiding subset-selection bias. The judge is not given gold answers, method names, or answer-correctness labels. It evaluates each retained reasoning chain as written, including whether the chain provides sufficient and coherent support for its own stated final answer. Since LLM-as-Judge evaluation over long reasoning chains is substantially more expensive than answer-only evaluation, we use these representative combinations rather than the full model–benchmark grid. The reported reasoning-quality evaluation costs approximately \$300 in OpenAI API usage.

Rubric. Each reasoning chain is scored on a 10–100 scale, in increments of 10, along four dimensions:

- **Completeness:** whether the chain contains a semantically sufficient derivation from the problem statement to the final answer.
- **Coherence:** whether the chain forms a smooth and logically connected line of reasoning, without abrupt jumps, contradictions, or disruptive interruptions.
- **Conciseness:** whether the chain avoids unnecessary repetition, redundant verification, and non-productive loops.
- **Justification:** whether a reader can understand why the final answer follows from the main derivation.

Few-shot calibration. We include six manually written in-context examples covering distinct quality patterns: complete and concise, complete but redundant, brief but sufficient, brief and incomplete, derivation with a corrected false start, and unresolved contradiction. All examples are based on a single math problem to avoid confounding rubric calibration with problem difficulty.

Evaluator bias control. To mitigate evaluator bias, the judge is not shown method names and evaluates all retained reasoning chains using the same rubric. The judge model is not used to train PUMA’s Redundancy Detector or to select stopping hyperparameters. Although the detector supervision uses LLM-generated redundancy annotations, it is constructed at the reasoning-step-pair level, whereas the judge evaluates complete retained reasoning chains; the two stages use different inputs, prompts, and objectives. To further validate judge reliability, we randomly sample 100 pairwise comparisons from the evaluation results. Each comparison contains two anonymized retained reasoning chains for the same question, and a human annotator independently selects the chain with higher overall explanation quality. The human judgments agree with GPT-5.4-thinking’s score-induced preferences in 85% of cases, suggesting that the judge-based relative comparisons are broadly aligned with human assessment. The absolute scores are modest across all methods, with the best average score in the mid-50s. This is because the rubric evaluates the quality of the retained reasoning chain, rather than final-answer correctness alone: a chain must be complete, coherent, concise, and sufficiently justified to receive a high score. A correct final answer is therefore not sufficient if the retained reasoning is incomplete, hard to follow, overly repetitive, or weakly justified.

Role:

You are an expert evaluator of reasoning chains. Your task is to evaluate how well a reasoning chain communicates a sufficient, productive derivation of the stated answer.

Core Principle:

Reward one clear successful line of reasoning. Do not reward extra exploratory branches, repeated verification, generic tutoring preambles, or exhaustive enumeration once the answer is already derivable.

Scoring Dimensions:

Score each chain from 10 to 100, in steps of 10, on: Completeness, Coherence, Conciseness, and Answer Justification.

Important Constraints:

Evaluate the reasoning chain as written. If the stated final answer is unsupported, contradicted, or does not follow from the reasoning, reflect this in Completeness and Answer Justification. Score Coherence and Conciseness independently. Do not use method names, gold answers, or external knowledge. Do not fill in missing proof steps with your own solution.

Rubric:

Completeness measures whether the chain contains a sufficient derivation from the problem to the stated final answer. Coherence measures whether the reasoning flow is logically connected, without abrupt jumps, contradictions, or unresolved self-corrections. Conciseness measures whether the chain avoids repetition, redundant verification, and non-productive loops. Justification measures whether a reader can understand why the stated final answer follows from the main derivation.

Few-shot Examples:

[The prompt includes six calibration examples covering concise sufficient reasoning, complete but redundant reasoning, brief but incomplete reasoning, false starts, and unresolved contradictions.]

Output Format:

Return only a JSON object with a brief rationale and four scores:

```
{"brief_rationale": "<1-2 sentences>",
"completeness": <10-100>, "coherence": <10-100>,
"conciseness": <10-100>, "justification": <10-100>}
```

Table 18: Judge prompt template for reasoning-chain quality evaluation. The full prompt includes per-dimension anchors and six calibration examples.

We therefore focus on relative comparisons across methods under the same anonymized judge and rubric.

F Internalizing PUMA into Model Weights: Implementation Details

This appendix provides full details for the internalization experiments reported in Section 6.2. All variants fine-tune DS-R1-Distill-Qwen-7B with LoRA (rank 64, alpha 128, all-linear) and are evaluated with pure vLLM inference, without any RD or AV at test time. Our RL training design is partly inspired by DEER [32].

Training data. We use the 12K problems from the Lightman split of MATH-benchmark [41], which is mathematics-only; consequently, MATH-500 performance can be regarded as in-distribution behavior, while the resulting models generalize zero-shot to AIME24 (harder math) and GPQA-D (cross-domain science). We verify that no problem in this train split overlaps with the MATH-500 test split, ensuring zero contamination of our in-distribution evaluation. For each problem, we generate one Full CoT trajectory from the base model and run the PUMA inference pipeline to record RD-flagged candidate exit positions and the verified stopping point.

Per-variant data construction. PUMA-SFT keeps rows $(Q, R_{\leq t^*}, A)$ where PUMA verifies an early exit at t^* and the regenerated answer is correct, filtered to aggressive compressions $(t^*/|R| < 0.6)$, yielding $\sim 6.5K$ examples. FixedExit-SFT replaces t^* with the earliest fixed-interval position ($K=3$) yielding a correct forced-stop answer; filtering is identical. PUMA-DPO pairs PUMA-truncated

chains (chosen) with Full CoT chains (rejected) when both are correct ($\sim 5.8\text{K}$ pairs). PUMA-RL and FixedExit-RL use a pre-expanded GRPO dataset where each row is a question concatenated with a reasoning prefix and the closing $\langle /think \rangle$ token, with the prefix truncated at a PUMA-flagged or fixed-interval position; the model is trained to generate only the answer phase that follows ($\sim 15\text{K}$ samples each). Standard-SFT and Standard-GRPO use no exit-position signal: Standard-SFT trains on the base model’s untruncated Full CoT trajectories, and Standard-GRPO performs standard GRPO on $(Q, \text{ground truth})$ pairs without any prefix conditioning, generating the entire $\langle think \rangle \dots \langle /think \rangle$ chain from scratch at each rollout.

Training. SFT runs for 3 epochs with learning rate 2×10^{-4} , batch size 1 with gradient accumulation 16, and `max_length` 16384. DPO runs for 1 epoch with the same learning rate, default β , and `rpo_alpha`=0.1 (an SFT anchor on the chosen response that we found necessary to prevent drift). GRPO runs for `max_steps`=1500 with learning rate 1×10^{-6} , `num_generations`=4 per group, `max_completion_length`=4096, and vLLM colocated generation. For Standard-GRPO, the reward is pure answer correctness. For PUMA-RL and FixedExit-RL, which generate only the answer phase after a provided reasoning prefix, we use the following reward:

$$R = r_{\text{correct}} \cdot (1.0 + 0.5 (1 - \ell / 4096)) + r_{\text{rank}}, \quad (6)$$

where $r_{\text{correct}} \in \{0, 1\}$ indicates answer correctness, ℓ is the completion length in tokens, and $r_{\text{rank}} \in \{+0.5, +0.25, 0, -0.25\}$ is a within-group rank bonus favoring the shortest correct trajectory among the four rollouts. All training uses bfloat16 with gradient checkpointing on $4 \times \text{NVIDIA GH200}$, implemented in ms-swift [91].